

软件工程与方法

第6讲　软件测试

主要内容

1. 软件缺陷
2. 软件测试的概念
3. 软件测试活动
4. 软件测试文档
5. 白盒测试技术
6. 黑盒测试技术
7. 针对环境和应用的测试
8. 软件测试的策略
9. 面向对象测试
10. 软件测试工具



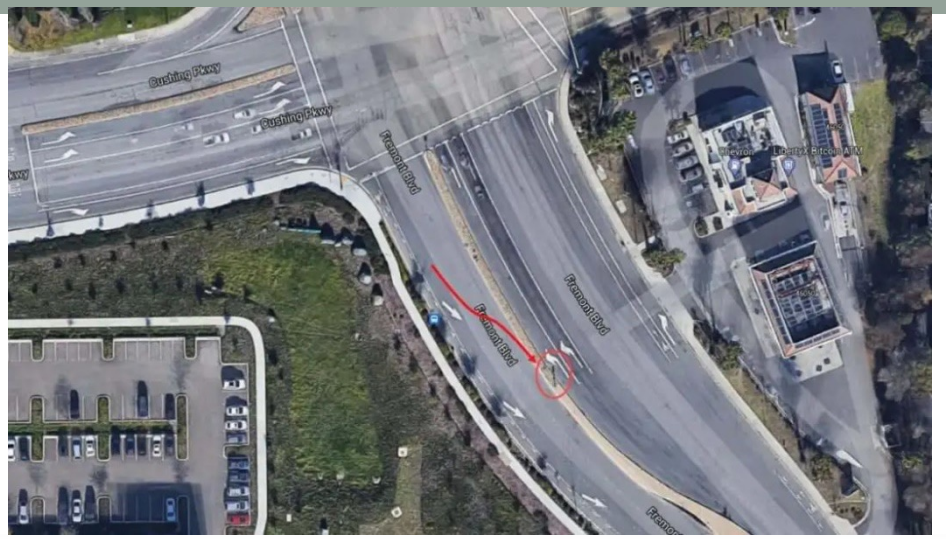
事故案例：阿丽亚娜5火箭事故





- 现象：1996年法国的阿丽亚娜5火箭首次发射时，发动机点火后37秒火箭爆炸，造成直接经济损失5亿美元，研制计划推迟一年。
- 原因：软件将表示为火箭水平速度的64位浮点数转换成16位有符号整数时，产生了溢出，引擎口偏转到极限状态。
- 工作失误：重用软件使用条件发生了变化，没有重新分析和验证。

AI发神经撞马路牙子！小马智行路试全无人自动驾驶结果「翻车」



Description of Defect :

Description of the Defect :	In a very rare combination of circumstances, one planning system diagnostic check could generate a "false positive" indication of a geolocation mismatch.
FMVSS 1 :	NR
FMVSS 2 :	NR
Description of the Safety Risk :	An ADS shutdown while the vehicle is moving could increase the risk of a collision.
Description of the Cause :	Former versions of certain ADS software included geolocation diagnostic coding that in rare circumstances could incorrectly interpret small floating-point number rounding errors (or "discrepancies"). As a result, it was possible that one specific diagnostic matching function could incorrectly interpret an inconsequential rounding discrepancy as a geolocation mismatch. During a test run, a Pony.ai AV was running a geolocation diagnostic check when a very

实例：谷歌程序员少打错了一个字符，导致大量Chromebook无法解锁设备

IT之家7月24日消息。近日，谷歌为Chromebook推送了Chrome OS 91版本，版本号91.0.4772.165。本次更新出现了重大Bug，导致许多用户的Chromebook无法解锁设备，官方已紧急撤销更新。

据用户反馈，在更新到该版本后，设备会卡在锁屏界面，无法进入桌面。即使用户输入了正确的密码，也无法使用，有的设备甚至会循环重启。

而开发者从谷歌官网源码中发现，该问题是由一个极其低级的错误引发的。从下图可以看到，开发者在if判断语句中，两个条件之间的与连接符本来是“&&”，但开发者粗心只打了一个“&”，造成程序的运行错误，系统无法对用户登录信息进行解密。

你认为用什么测试方法能发现这个bug?

（续）

Chromium Gerrit CHANGES DOCUMENTATION BROWSE

3039560 cryptohome: Fix conditional statement in VaultKeyset cryptohome/vault_keyset.cc

Base gitiles → Patchset 2 gitiles DOWNLOAD

File

```
461   google::protobuf::uint8* buf =
462       static_cast<google::protobuf::uint8*>(contents.data());
463   serialized.SerializeWithCachedSizesToArray(buf);
464
465   bool ok = platform_->WriteFileAtomicDurable(filename, contents,
466                                               kVaultFilePermissions);
467   return ok;
468 }
469
470 std::string VaultKeyset::GetLabel() const {
471   if (key_data_.has_value() & !key_data_->label().empty()) {
472     return key_data_->label();
473   }
474   // Fallback for legacy keys, for which the label has to be inferred from the
475   // index number.
476   return base::StringPrintf("%s%d", kKeyLegacyPrefix, legacy_index_);
477 }
478
479 bool VaultKeyset::IsLECredential() const {
480   if (key_data_.has_value()) {
481     return key_data_->policy().low_entropy_credential();
```

File

+460 common lines +10

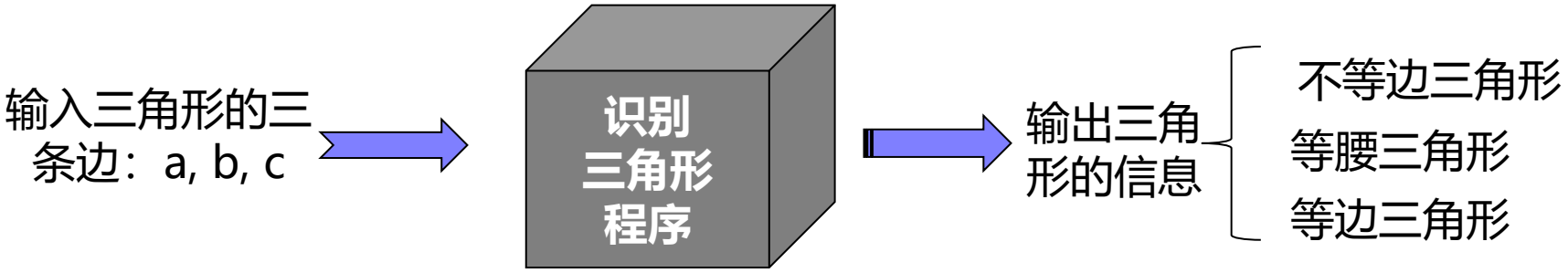
```
461   google::protobuf::uint8* buf =
462       static_cast<google::protobuf::uint8*>(contents.data());
463   serialized.SerializeWithCachedSizesToArray(buf);
464
465   bool ok = platform_->WriteFileAtomicDurable(filename, contents,
466                                               kVaultFilePermissions);
467   return ok;
468 }
469
470 std::string VaultKeyset::GetLabel() const {
471   if (key_data_.has_value() && !key_data_->label().empty()) {
472     return key_data_->label();
473   }
474   // Fallback for legacy keys, for which the label has to be inferred from the
475   // index number.
476   return base::StringPrintf("%s%d", kKeyLegacyPrefix, legacy_index_);
477 }
478
479 bool VaultKeyset::IsLECredential() const {
480   if (key_data_.has_value()) {
481     return key_data_->policy().low_entropy_credential();
```

+529 common lines +10

课堂练习题

- 设计测试用例

  Myers用以下程序对测试能力进行自我评估：某程序读入3个数值，这3个数值表示三角形的三条边。该程序打印信息表明三角形是不等边三角形、等腰三角形或等边三角形，编写测试用例集测试该程序。



软件测试要解决的基本问题

软件测试的本质是通过系统性方法平衡质量与成本，其核心挑战在于将有限的资源聚焦于高风险领域，同时通过技术手段（如自动化、AI）提升效率和覆盖率。

① Test Oracle问题：如何判断测试结果正确性

② 测试充分性：如何覆盖所有关键场景

③ 测试有效性：如何高效发现缺陷

④ 测试停止标准：何时终止测试

⑤ 测试环境与数据管理

⑥ 缺陷管理与协作效率

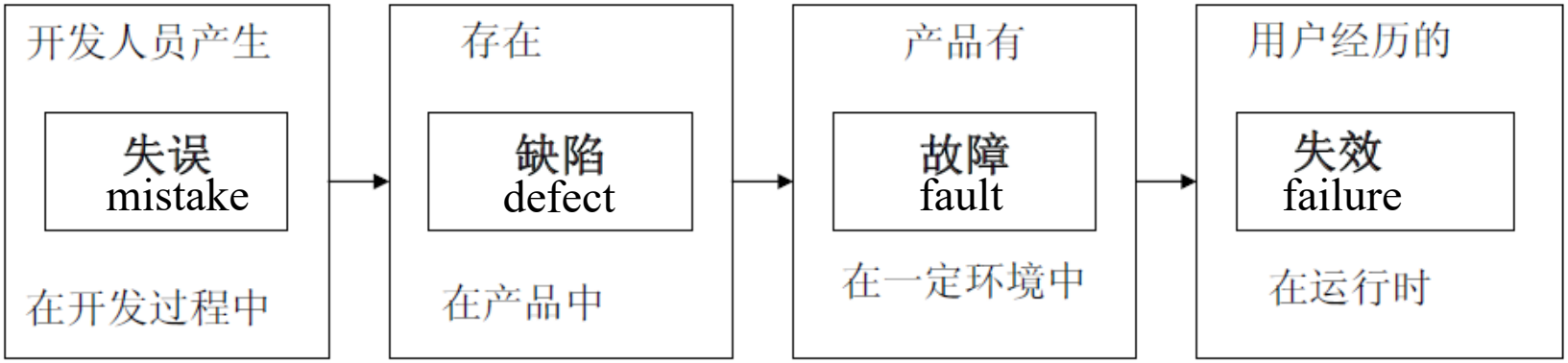
软件测试

1 软件缺陷



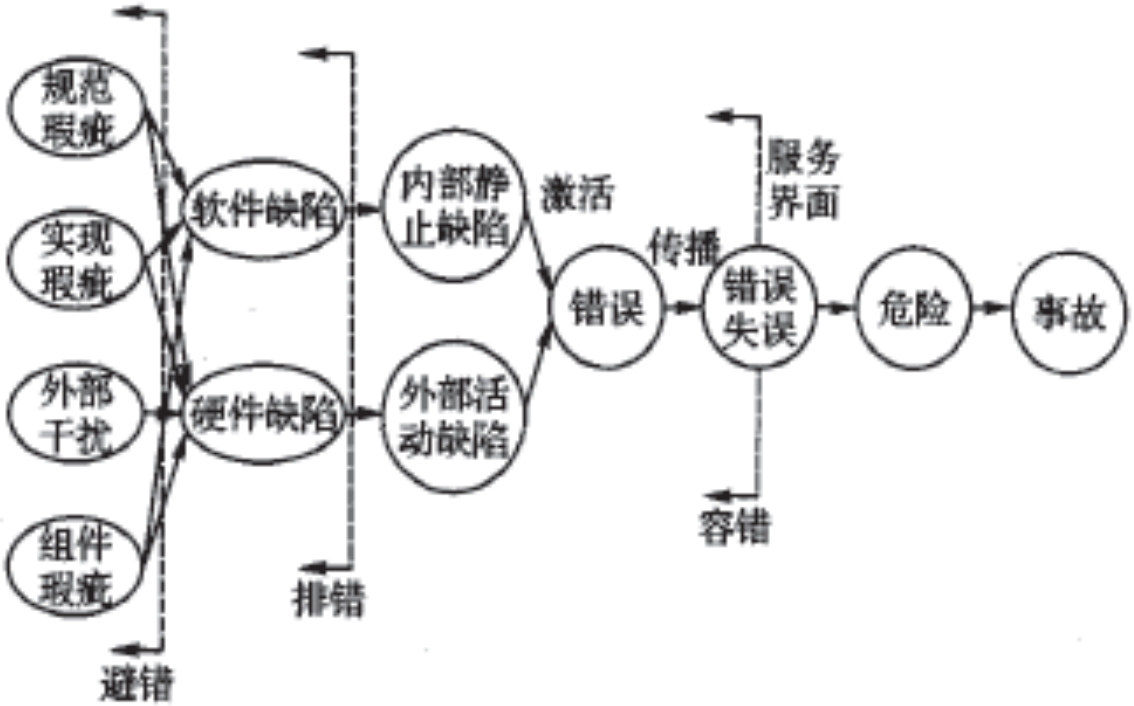


软件为什么会失效?



- 人的因素——错误的源头
  - 软件是人写的，人的认知局限导致失误是失效的第一因。
- 技术的因素——缺陷的温床
  - 失误最终要沉淀到代码和技术实现中，形成缺陷。
- 过程的因素——缺陷的放大器
  - 过程失效，没有拦截住缺陷，缺陷漏到生产环境的最后一道防线失守。
- 环境的因素——失效的催化剂
  - 有些失效，代码本身没问题，但在特定环境下被触发。

缺陷、错误及失效之间的关系



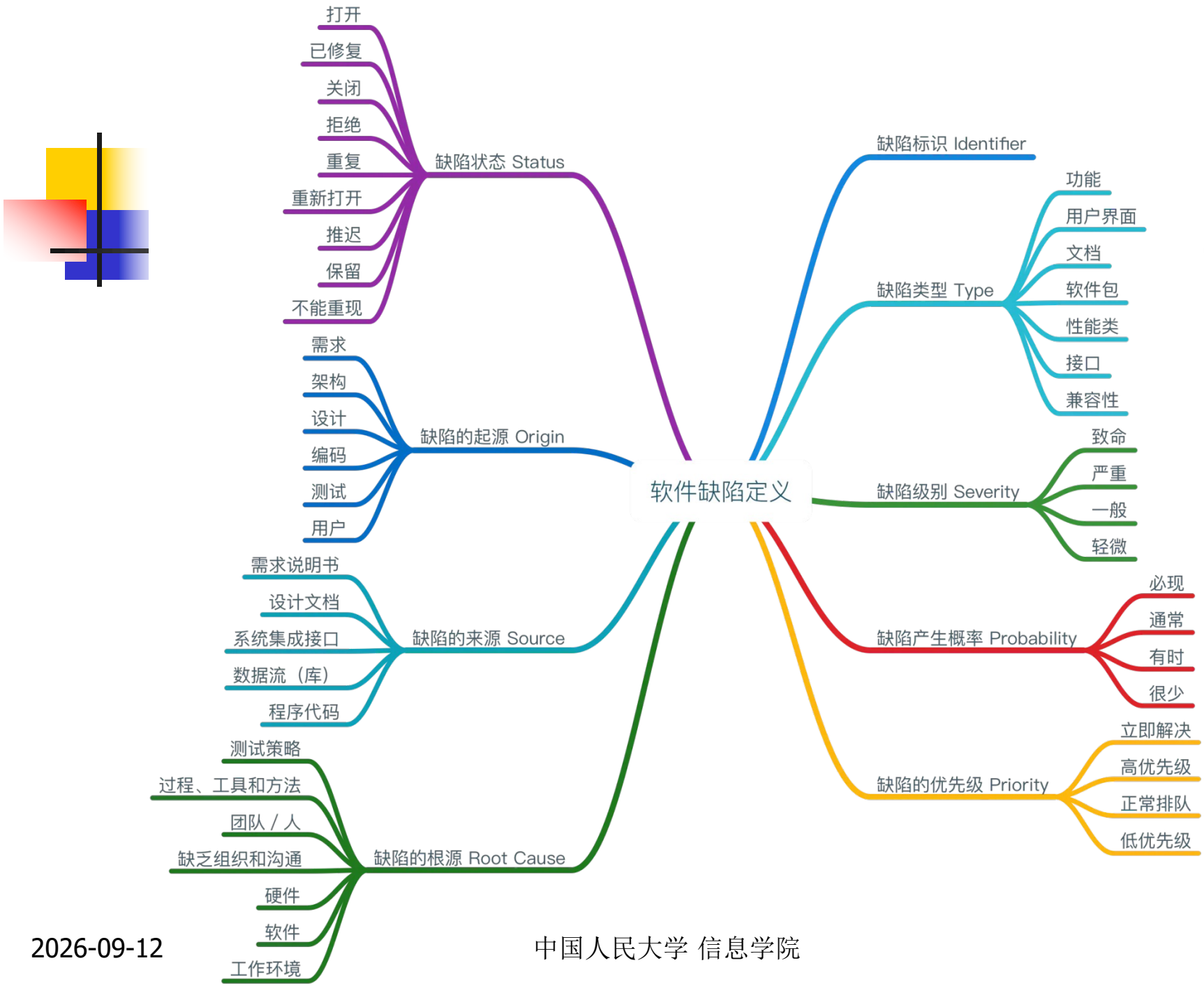
- 缺陷（Fault）
- 错误（Error）
- 失效（Failure）

软件缺陷

- **软件缺陷**（bug），即为计算机软件或程序中存在的某种破坏正常运行能力的问题、错误，或者隐藏的功能缺陷。
- 缺陷的存在会导致软件产品在某种程度上不能满足用户的需要。

IEEE729-1983对缺陷有一个标准的定义：

从产品内部看，缺陷是软件产品开发或维护过程中存在的错误、毛病等各种问题；从产品外部看，缺陷是系统所需要实现的某种功能的失效或违背。


软件缺陷定义
缺陷状态 Status
打开
已修复
关闭
拒绝
重复
重新打开
推迟
保留
不能重现
缺陷的起源 Origin
需求
架构
设计
编码
测试
用户
缺陷的来源 Source
需求说明书
设计文档
系统集成接口
数据流（库）
程序代码
缺陷的根源 Root Cause
测试策略
过程、工具和方法
团队 / 人
缺乏组织和沟通
硬件
软件
工作环境
缺陷标识 Identifier
缺陷类型 Type
功能
用户界面
文档
软件包
性能类
接口
兼容性
缺陷级别 Severity
致命
严重
一般
轻微
缺陷产生概率 Probability
必现
通常
有时
很少
缺陷的优先级 Priority
立即解决
高优先级
正常排队
低优先级

缺陷属性（bug attribute）

① **缺陷标识(Identifier)：** 缺陷标识是标记某个缺陷的一组符号。每个缺陷必须有一个唯一的标识。

② **缺陷类型 (Type)：** 缺陷类型是根据缺陷的自然属性划分的缺陷种类。

③ **缺陷严重程度 (Severity) ：** 缺陷严重程度是指因缺陷引起的故障对软件产品的影响程度。

④ **缺陷优先级(Priority)：** 缺陷的优先级指缺陷必须被修复的紧急程度。

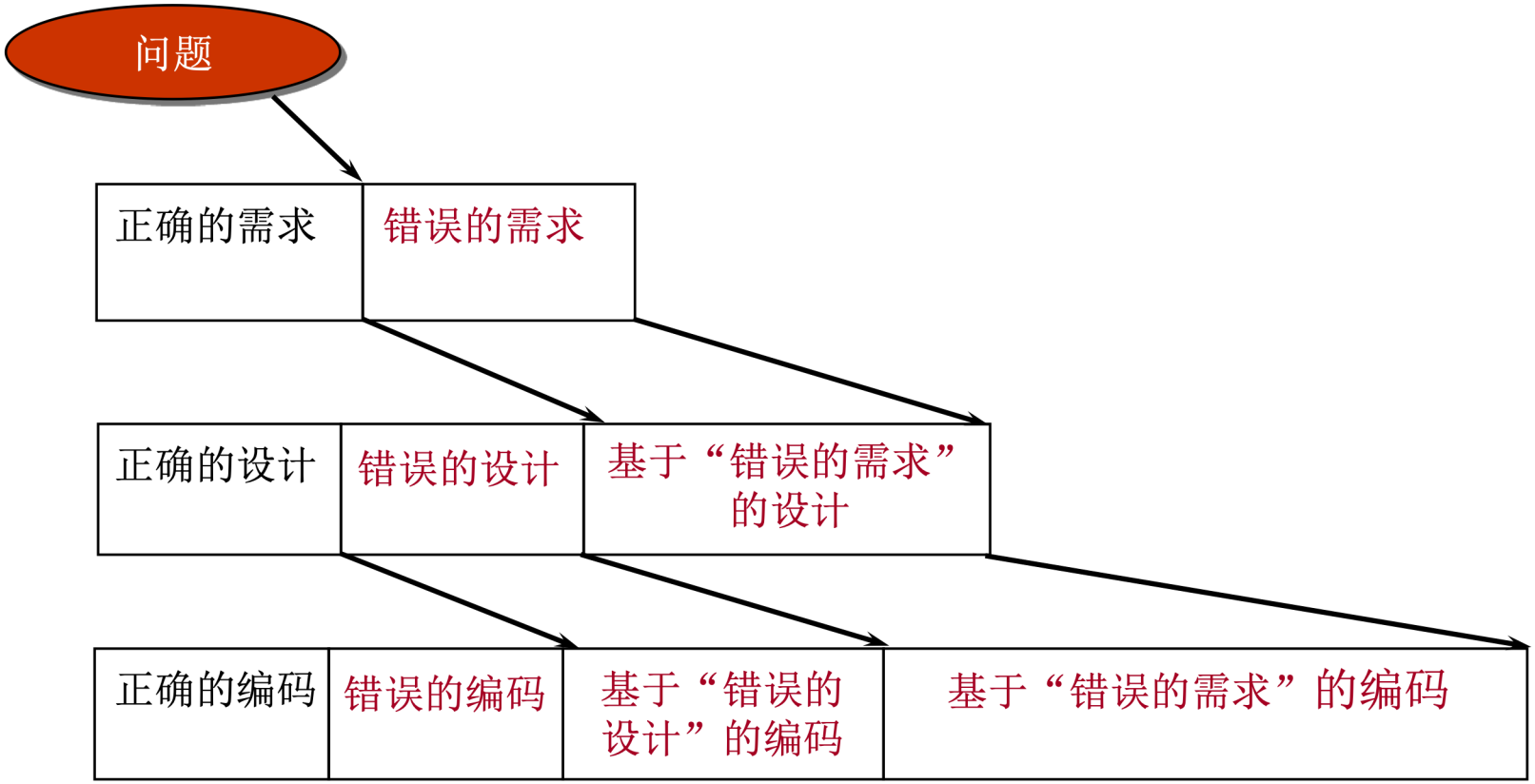
⑤ **缺陷状态(Status) ：** 缺陷状态指缺陷通过一个跟踪修复过程的进展情况。

⑥ **缺陷起源(Origin) ：** 缺陷来源指缺陷引起的故障或事件第一次被检测到的阶段。

⑦ **缺陷来源(Source)：** 缺陷来源指引起缺陷的起因。

⑧ **缺陷根源(Root Cause)：** 缺陷根源指发生错误的根本因素。

“错误的需求”的扩散效应


问题
正确的需求
错误的需求
正确的设计
错误的设计
基于“错误的需求”的设计
正确的编码
错误的编码
基于“错误的设计”的编码
基于“错误的需求”的编码

水波效应（Water wave effect ）

- **水波效应**：指人们对程序的A处进行修改时引出B处出错误，修改B处时又影响到C处，以此类推形成的一石激起千层浪的连带影响的局面。
- 在软件维护过程中，由一个模块的修改而导致隐含缺陷、错误的放大以及一连串新错误出现，形成“水波效应”。



软件中的Bug数量

- SonarQube 2025年报告：根据其对超过79亿行代码的分析，平均0.21 KLOC可靠性问题（Bug）。
- Coverity 2013年报告：在分析超过7.5亿行开源代码时，发现其平均缺陷密度为0.59 KLOC。
- 根据《2025年中国软件行业基准数据（CSBMK®-202510）》，国内软件行业的缺陷密度中值为4.6 KLOC（0.23/50行Java代码×1000），交付质量（交付后6个月内发现的缺陷数）中值为9.3 KLOC。

经典研究参考：

- NASA数据：在航天飞机软件项目中，缺陷密度被控制在<0.1 KLOC，甚至在某些版本中达到<0.01。
- IBM数据：在大型主机系统研究中，发布后的缺陷密度通常在0.5~2.0 KLOC。
- 开源研究：针对Apache或Linux的研究显示，随着版本成熟，缺陷密度会从早期的>5.0 KLOC迅速下降到<0.5 KLOC。

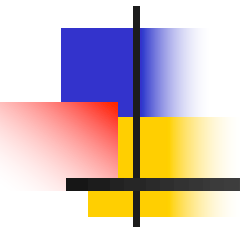
Bug密度行业基准值

根据Capers Jones的研究（软件工程领域权威专家），不同规模项目的典型缺陷密度（每千行代码缺陷数）如下：

- 小型项目（<2000行代码）：缺陷密度为0-25 KLOC（如工具类小程序，逻辑简单，Bug数量少）；
- 中型项目（2000-16000行代码）：缺陷密度为0-40 KLOC（如企业级管理系统，模块较多，Bug数量中等）；
- 大型项目（16000-64000行代码）：缺陷密度为0.5-50 KLOC（如电商平台、ERP系统，复杂度高，Bug数量较多）；
- 超大型项目（>512000行代码）：缺陷密度为4-100 KLOC（如操作系统、数据库，规模庞大，Bug数量极多）。
- 例如，一个10万行代码的中型项目，按20个/千行的缺陷密度计算，约有2000个Bug；而一个100万行代码的超大型项目，按50个/千行的缺陷密度计算，约有50000个Bug。

动态Bug密度基准值

- 行业基准：
  - 一个被广泛认可的行业标准是，高质量软件的缺陷密度应控制在1.0 KLOC以下 。成熟项目的缺陷密度通常在1.5~2.5 KLOC之间 。
- 不同阶段的差异：
  - 开发过程中的总缺陷：一个项目在开发、测试各阶段发现的总缺陷密度可能较高。例如，有案例统计一个3万行代码的项目，总缺陷数达319个，缺陷密度约为10.6 KLOC 。
  - 发布后的逃逸缺陷：发布后用户发现的缺陷密度则要低得多。CMMI三级企业的目标通常是0.5~1.5 KLOC，四级企业则要求低于0.5 KLOC。
  - 个人开发者水平：有更早的统计表明，未经训练的软件工程师引入的缺陷密度可能高达50 KLOC 。



软件测试

2 软件测试的概念

问题

为什么要进行软件测试?

软件测试是为了发现软件产品所存在的软件缺陷（bug），从而纠正（fix）这些软件缺陷，使软件系统更好地满足用户的需求。

证明	检测	预防
表明软件能够工作	发现错误	管理质量
20世纪60年代	20世纪70年代中期	20世纪90年代

软件测试（Software Testing）的定义

- ***Grenford J. Myers***:
  - 测试是为了发现错误而执行一个程序或系统的过程。
- ***传统：***
  - 软件测试描述一种旨在促进鉴定软件的正确性、完整性、安全性和质量的过程。
  - 软件测试是一种实际输出与预期输出之间的审核或者比较过程。
- ***IEEE***:
  - 测试是使用人工和自动手段来运行或检测某个系统的过程，其目的在于检验系统是否满足规定的需求或弄清预期结果与实际结果之间的差别。

测试目的

证明程序的可信性

- Grenford J. Myers提出的测试目的
  - 测试是程序的执行过程，目的在于发现错误；
  - 一个好的测试用例在于能够发现至今未发现的错误；
  - 一个成功的测试是发现了至今未发现的错误的测试。
- 测试的其它目的
  - 测试并不仅仅是为了找出错误。通过分析错误产生的原因和错误的发生趋势，可以帮助项目管理者发现当前软件开发过程中的缺陷，以便及时改进。这种分析也能帮助测试人员设计出有针对性的测试方法，改善测试的效率和有效性。
  - 没有发现错误的测试也是有价值的，完整的测试是评定软件质量的一种方法。
  - 根据测试目的的不同，还有回归测试、压力测试、性能测试等，分别为了检验修改或优化过程是否引发新的问题、软件所能达到处理能力和是否达到预期的处理能力等。

ISTQB软件测试原则

- 原则一：测试证明软件存在缺陷　Testing shows presence of defects
- 原则二：穷尽测试是不可能的　Exhaustive testing is impossible
- 原则三：尽早介入测试　Testing Early
- 原则四：缺陷具有集群性　Defect Clustering
- 原则五：杀虫剂悖论　Pesticide Paradox
- 原则六：测试是上下文相关的　Testing is context dependent
- 原则七：无错误谬论　Absence of error - Fallacy

ISTQB （International Software Testing Qualifications Board，国际软件测试资质认证委员会）是国际唯一全面权威的软件测试资质认证机构，主要负责制订和推广国际通用资质认证框架，即国际软件测试资质认证委员会推广的软件测试工程师认证(ISTQB Certified Tester) 项目。

软件测试的原理

(1) **测试可以证明缺陷存在，但不能证明缺陷不存在**

(2) **穷尽测试是不可能的**

- 考虑所有可能输入值和它们的组合，并结合不同的测试前置条件进行穷尽测试是不可能的。

(3) **测试活动应当尽早进行**

- 在软件生命周期中，测试活动应当尽早开始，早发现缺陷，能够避免缺陷向后传递和放大，有利于降低成本。

(4) **缺陷集群性**

- 通常情况下，缺陷是集群分布的。因此，如果在一个地方发现了很多缺陷，那么通常在附近会有更多的缺陷。

(5) **杀虫剂悖论**

- 如果同样的测试用例被重复执行，会减少其有效性。为了维持测试的有效性，应对测试用例进行不断修改和更新。

(6) 测**试依赖于测试上下文**

- 测试必须与应用程序的运行环境和使用中固有的风险相适应。因此，没有两个系统可以以完全相同的方式进行测试。对于每一个软件系统，测试出口准则等应当根据它们使用的环境量身定制。

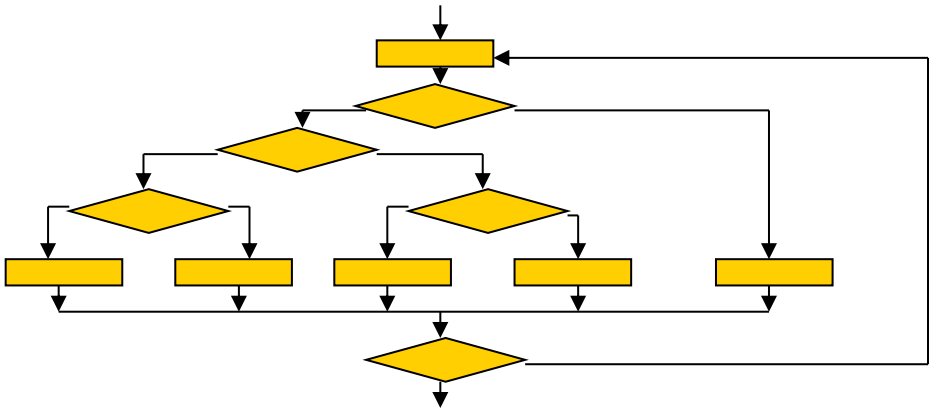
(7) **没有失效就是有用系统是一种谬论**

- 没有失效并不能保证系统就能满足用户需求，因为用户需求可能存在不明确和变化可能。

(8) **测试为全软件生命周期活动**

为什么?

穷举测试是不可能的！

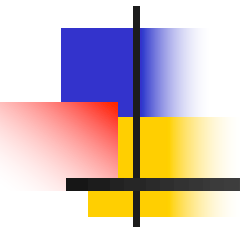


考虑20次的循环，有四个 IF-THEN-ELSE 结构，则执行的路径可达$5^{20}\approx10^{14}$条。设每个路径测试需1毫秒，一天工作24小时，一年365天，则需3170年。

软件可测试性（testability）



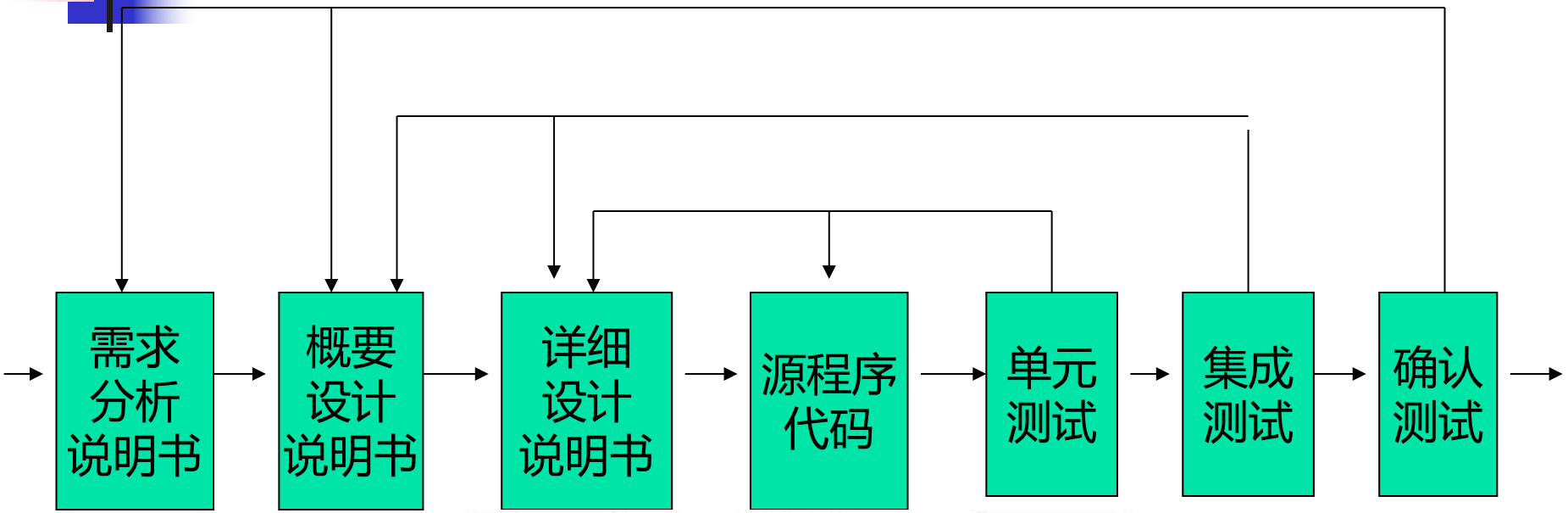
- 可操作性
  Operability—it operates cleanly
- 可观察性
  Observability—the results of each test case are readily observed
- 可控制性
  Controlability—the degree to which testing can be automated and optimized
- 可分解性
  Decomposability—testing can be targeted
- 简单性
  Simplicity—reduce complex architecture and logic to simplify tests
- 稳定性
  Stability—few changes are requested during testing
- 易理解性
  Understandability—of the design



软件测试

3 软件测试活动

测试与软件开发各阶段的关系

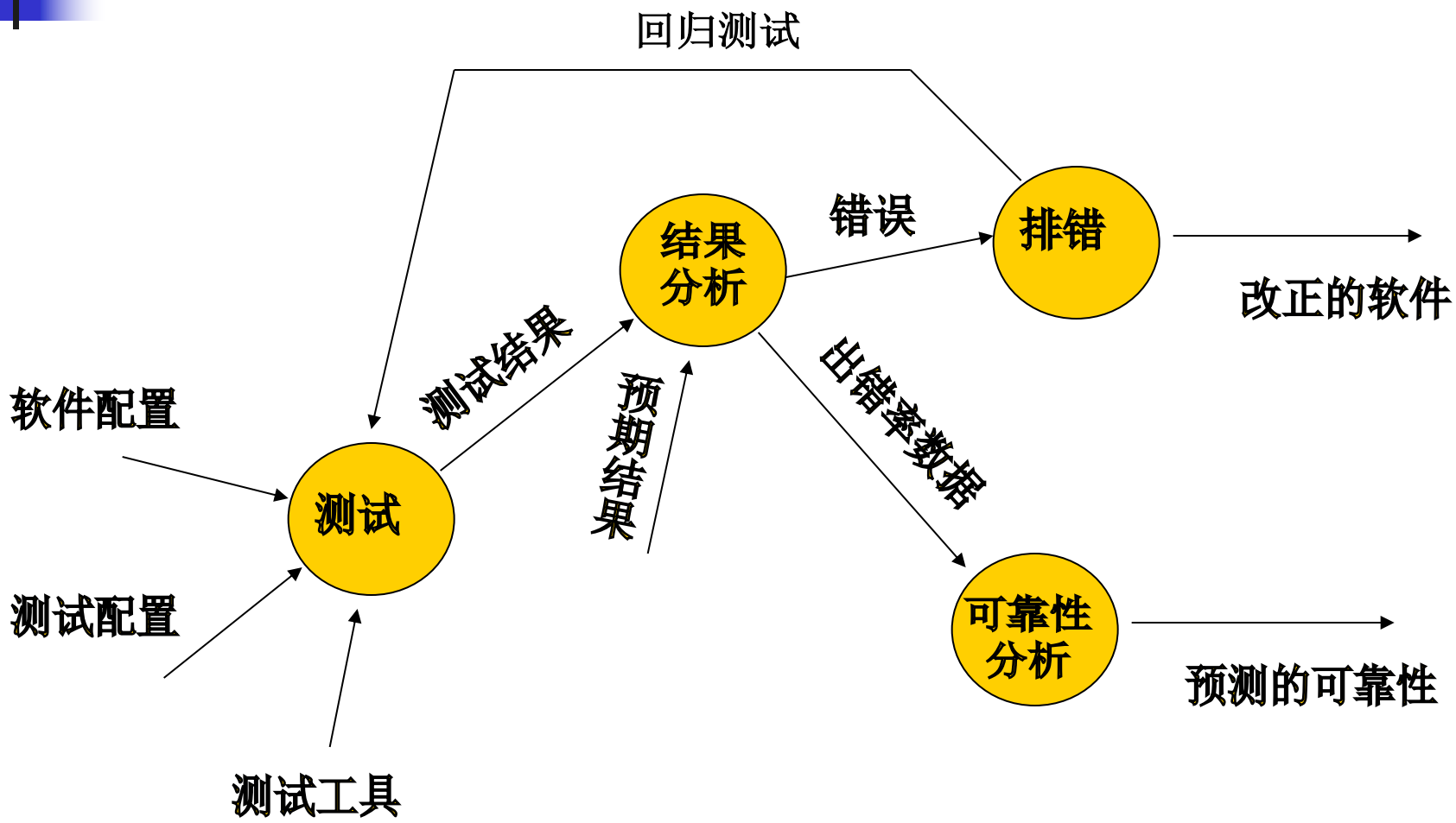




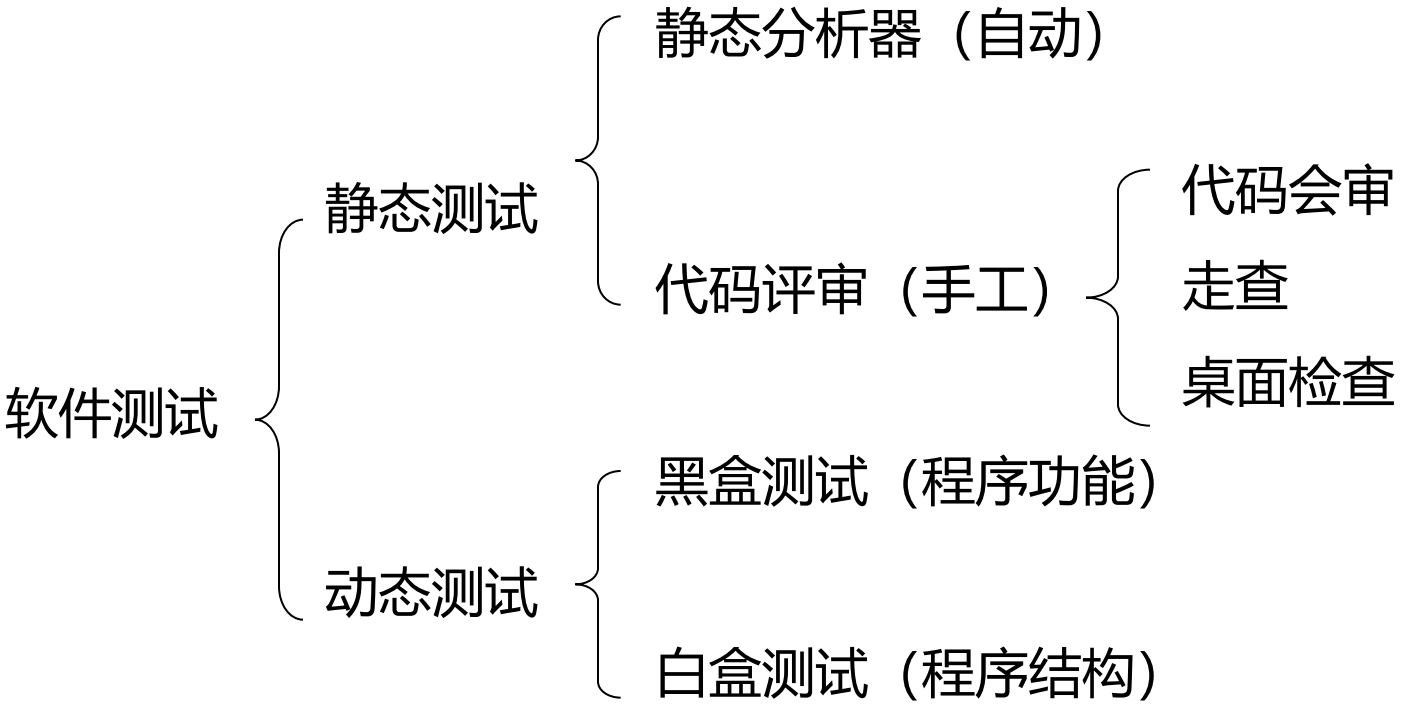
软件测试活动

需求阶段	设计阶段	实现阶段	测试阶段	验收阶段
• 用例情景测试 • 原型走查 • 模型走查 • 需求评审	• 模型走查 • 原型走查 • 设计评审	• 代码走查 • 接口分析 • 文档评估		
• 制定测试计划	• 制定测试计划 • 测试设计	• 编写测试用例 • 制定测试过程 • 单元测试	• 制定测试过程 • 集成测试 • 系统测试	• α 测试 • β 测试 • 验收测试
回归测试，质量保证				

软件测试信息流


回归测试
结果分析
错误
排错
改正的软件
测试结果
预期结果
出错率数据
软件配置
测试
测试配置
测试工具
可靠性分析
预测的可靠性

软件测试类型

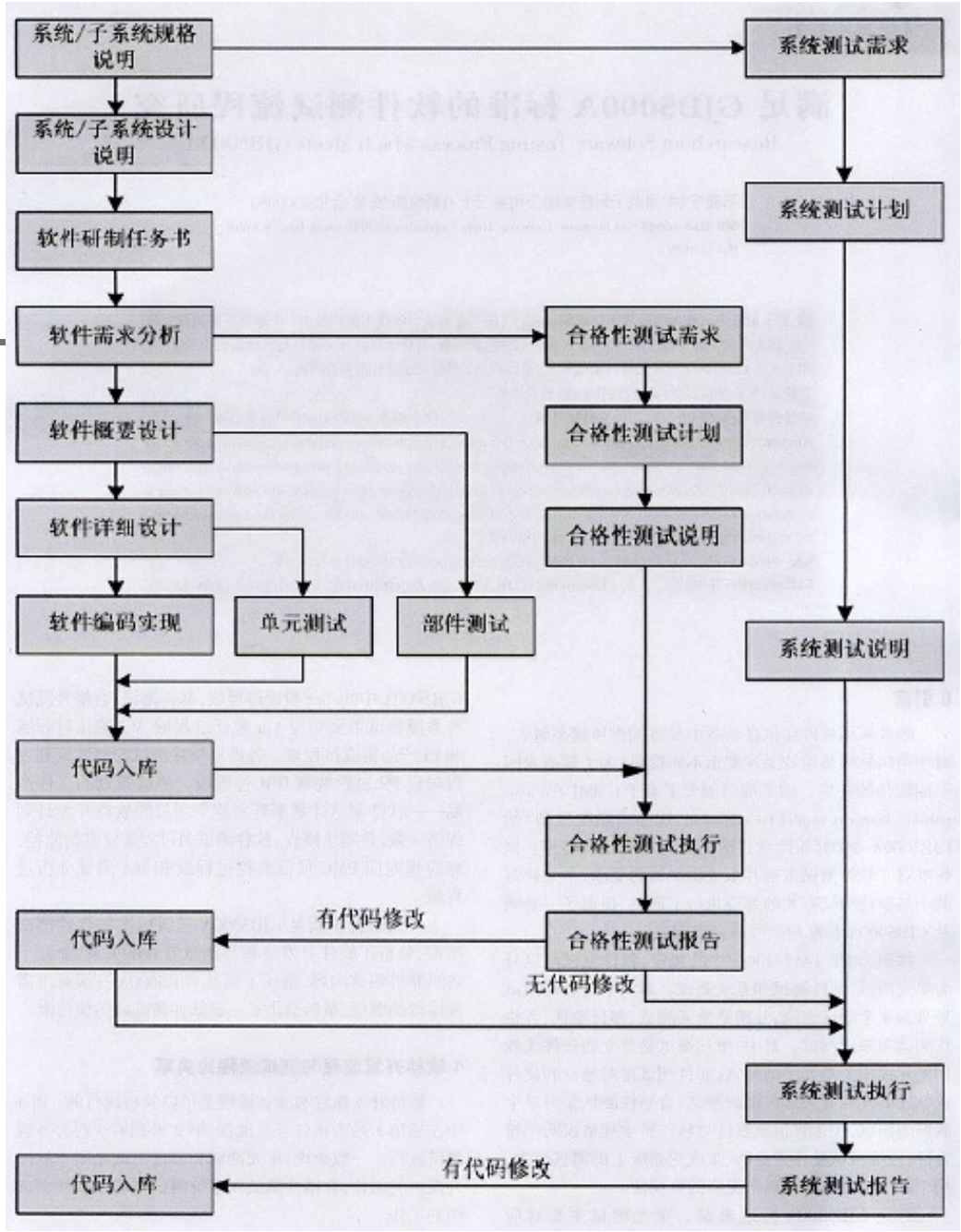

软件测试
静态测试
静态分析器（自动）
代码评审（手工）
代码会审
走查
桌面检查
动态测试
黑盒测试（程序功能）
白盒测试（程序结构）

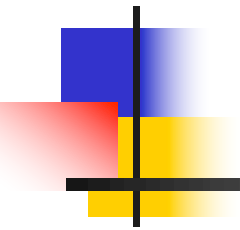
代码检查

代码检查（code Inspection）是最正式的审查类型，它是一种**静态测试**，以避免后期的缺陷倍增。

- 代码检查的主要目的是发现缺陷，也可以发现任何流程改进（如果有）。
- 检查报告列出了调查结果，其中包括可用于帮助改进流程以及纠正审查文件中缺陷的指标。
- 会议前的准备工作至关重要，包括阅读任何源文件以确保一致性。
- 检查通常由受过培训的主持人领导，**他不是代码的作者**。
- 检查流程是基于规则和检查表的**最正式的审查类型**，并利用进入和退出标准。
- 它通常涉及代码的同行检查，每个人都有一组定义的角色。
- 会后，采用正式的后续流程，以确保及时完成纠正措施。

软件测试过程与开发过程并行示意图





软件测试

4 软件测试文档

测试计划（*Test Plan*）

- 作用：
  - 是测试工作的指导性文档，规定测试活动的范围、方法、资源和进度；
  - 明确正在测试的项目、要测试的特性、要执行的测试任务、每个任务的负责人，以及与计划相关的风险。
- 主要内容：
  - 测试目标、测试方法、测试范围、测试资源、测试环境和工具、测试体系结构、测试进度表

测试规范（*Test Specification*）

- 作用：
  - 是从整体上规定测试案例的运行环境、测试方法、生成步骤、执行步骤以及调试和验证的步骤。
- 内容：
  - 系统运行环境、总体测试方法、测试用例的生成步骤、测试用例的执行步骤、调试和验证。

测试用例（*Test Case*）

- 测试用例，是指对一项特定的软件产品进行测试任务的描述，体现测试方案、方法、技术和策略。基本内容应当包含前置条件、测试步骤、预期结果，在测试用例执行后应当包含实际结果、测试结果、执行时间、执行人。有时候也需要加上数据准备、测试环境、测试脚本等等。
- 作用：
  - 是数据输入和期望结果组成的对；
  - 指出使用具体测试案例产生的测试程序的任何限制。
- 测试用例可以被组织成一个测试系列，用来实现某个特定的测试目。例如：
  - 测试兼容性、安全性、可用性等
  - 测试特定功能

系统名称			版本		
用例标识			模块名		
设计者		时间		测试方法	
用例简介					
测试步骤					
测试输入					
测试输出或预期结果					
测试通过标准					

测试用例模版

缺陷报告（*Bug Report*）

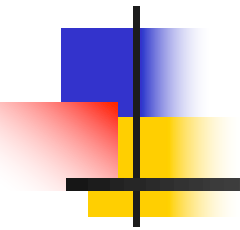
- Bug，为程序中错误或异常，准确的含义是不符合需求的功能点。
- 作用：
  - 编写在需要调查研究的测试过程期间发生的任何事件。
  - 简而言之，就是记录软件缺陷。
- 主要内容：
  - 缺陷编号、题目、状态、提出、解决、所属项目、测试环境、缺限报告步骤、期待结果、附件
- 在报告缺陷时，要讲明缺陷的严重性和优先级。
  - 严重性表示软件的恶劣程度，反映对产品和用户的影响。
  - 优先级表示修复缺陷的重要程度和应该何时修复。

示例：缺陷类型

Defect Name	Description
Documentation	Comments, software unit description
Build, package	Change management, library, version control
Assignment	Declaration, duplicate name, scope, limit
Design Entities Interface	Procedure calls and reference, I/O, user formats
Checking	Error message, inadequate checks
Data	Structure, content
Function	Logic, pointer, loop, recursion, computation, function
System Behavioral	Configuration, timing, memory
Performance	Metric values for measurable attributes of the system, execution time, timing of event, etc,
Quality	Attributes of the system: usability, portability, reliability, maintainability
Constraints	Constraints under which the system must operate usually affected by policies or regulations such as security, safety, or restriction hardware, etc…
Software Interface	Data management systems, operating system, other applications
User Interface	Characteristics for the human/computer interaction: screen format, validation for user input, function availability, page layout, etc…
Norms	Notation for design entities representation

测试报告（*Test Report*）

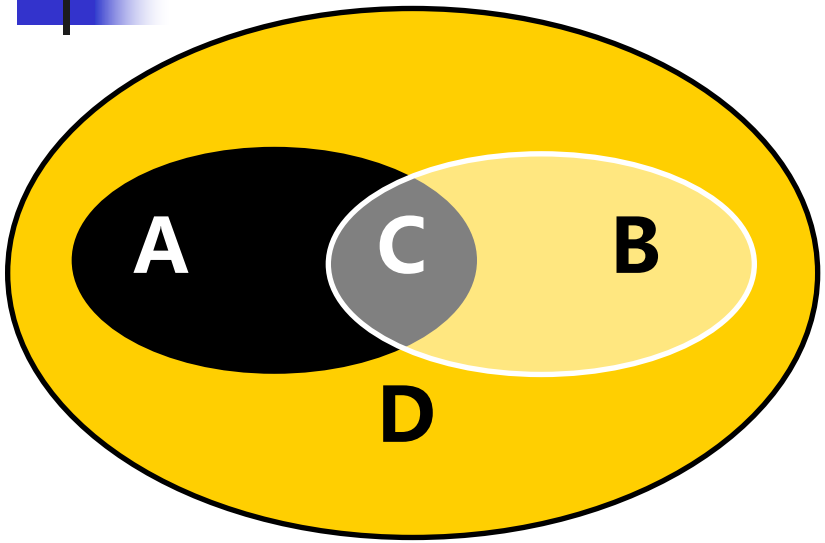
- 作用
  - 测试报告是指把测试的过程和结果写成文档，对发现的问题和缺陷进行统计分析。
  - 为纠正软件的存在的质量问题提供依据，同时为软件验收和交付打下基础。
- 内容
  - 引言（目的、背景、缩略语、参考文献）
  - 测试概要(测试方法、范围、测试环境、工具)
  - 测试结果与缺陷分析（功能、性能）
  - 测试结论与建议（项目概况、测试时间 测试情况、结论性能汇总）
  - 附录（缺陷统计）



软件测试

5 白盒测试技术

黑盒测试与白盒测试关系



白盒测试比黑盒测试的理论性更强，测试难度更大。

A-只能用黑盒测试发现的错误
B-只能用白盒测试发现的错误
C-两种方法都能发现的错误
D-两种方法都不能发现的错误

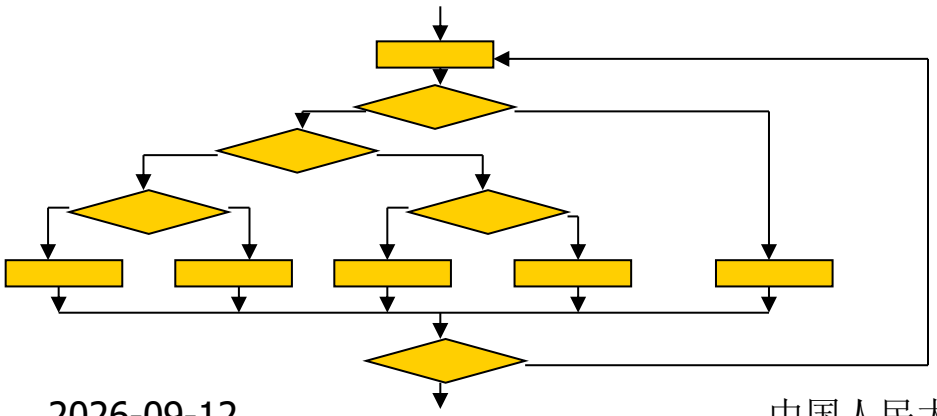
黑盒测试是在软件界面上的测试，是为了发现软件错误，但更常用于：

- 证实软件功能的可操作性；
- 证实能很好地接受输入，并正确的产生输出；
- 证实对外部信息完整性的保持。

白盒测试是对程序控制结构的测试，它依赖对程序细节的了解，运用特定条件和与循环集的测试用例，对软件逻辑路径进行测试，在不同点检验“程序的状态”以判定预期状态或待验证状态与真实状态是否相符。

白盒测试（*White Box Testing*）

- 又称结构测试，逻辑驱动测试，基于程序测试。它把测试对象看做一个透明的盒子（glass box），它允许测试人员利用程序内部的逻辑结构及有关信息，设计或选择测试用例，对程序所有逻辑路径进行测试。
- 问题：对一个具有多重选择和循环嵌套的程序，不同的路径数目可能是天文数字。



考虑20次的循环，有四个 IF-THEN-ELSE 结构，则执行的路径可达$5^{20} \approx 10^{14}$条。设每个路径测试需1毫秒，一天工作24小时，一年365天，则需3170年

白盒测试的特点

白盒测试定义：

基于系统或者组件的内部实现结构和逻辑，寻找缺陷的测试技术。

白盒测试的原则

- 保证一个模块中的所有独立路径至少被测试一次。
- 对所有的逻辑判定均需测试取真和取假两种情况。
- 在上下边界及可操作范围内运行所有循环。
- 检查程序的内部数据结构，保证其结构的有效性。

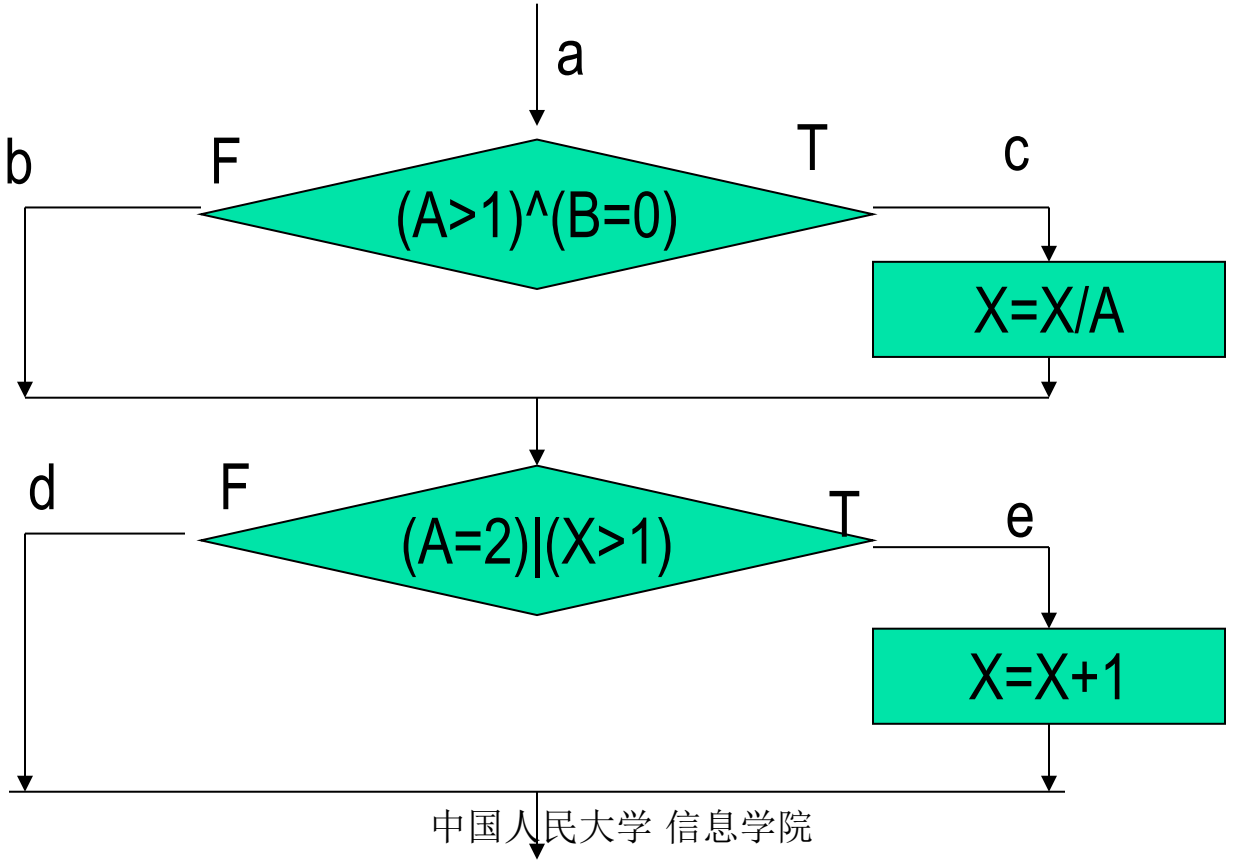
- 优点：
  - 代码覆盖率高
- 缺点：
  - 覆盖所有代码路径难度大；
  - 业务功能可能覆盖不全；
  - 测试开销大。

白盒测试方法分类：

- 静态：桌面检查、代码审查、代码走查、代码扫描工具
- 动态：逻辑覆盖法，通过对程序逻辑结构的遍历实现程序的覆盖，包括：语句覆盖、判断覆盖、条件覆盖、判定条件覆盖、条件组合覆盖、路径覆盖、基本路径测试法。

白盒测试用例设计

- 逻辑覆盖测试方法——解决控制测试问题



路径：

L1(a c e)=((A>1)and(B=0))and((A=2)or(X>1))

=(A>1)and(B=0)and(A=2)or(A>1)and(B=0)and(X>1)

=(A=2)and(B=0)or(A>1)and(B=0)and(X>1)

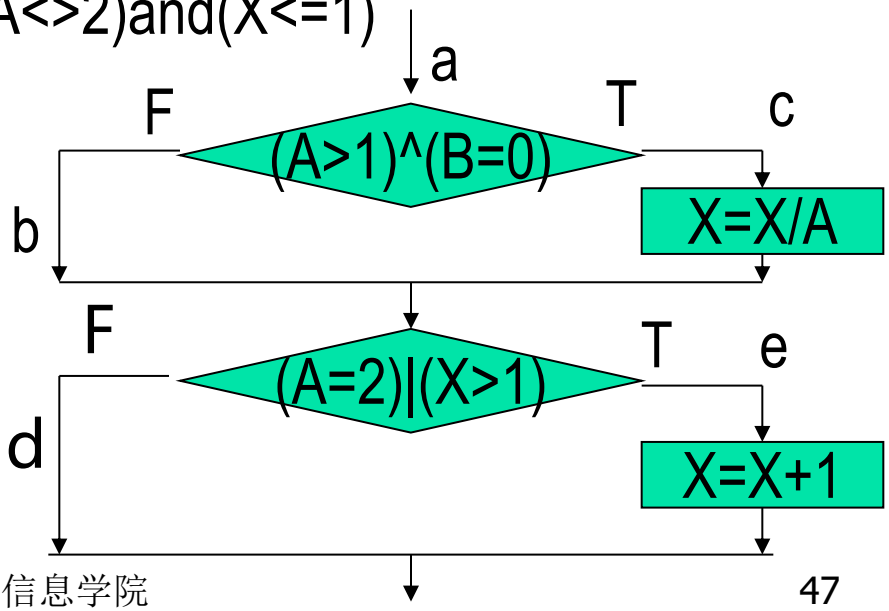
L2(a b d)=(A<=1)and(X<=1)or(B<>0)and(A<>2)and(X<=1)

L3(a b e)=(A<=1)and(X>1)or(B<>0)

and(A=2)or(B<>0)and(X>1)

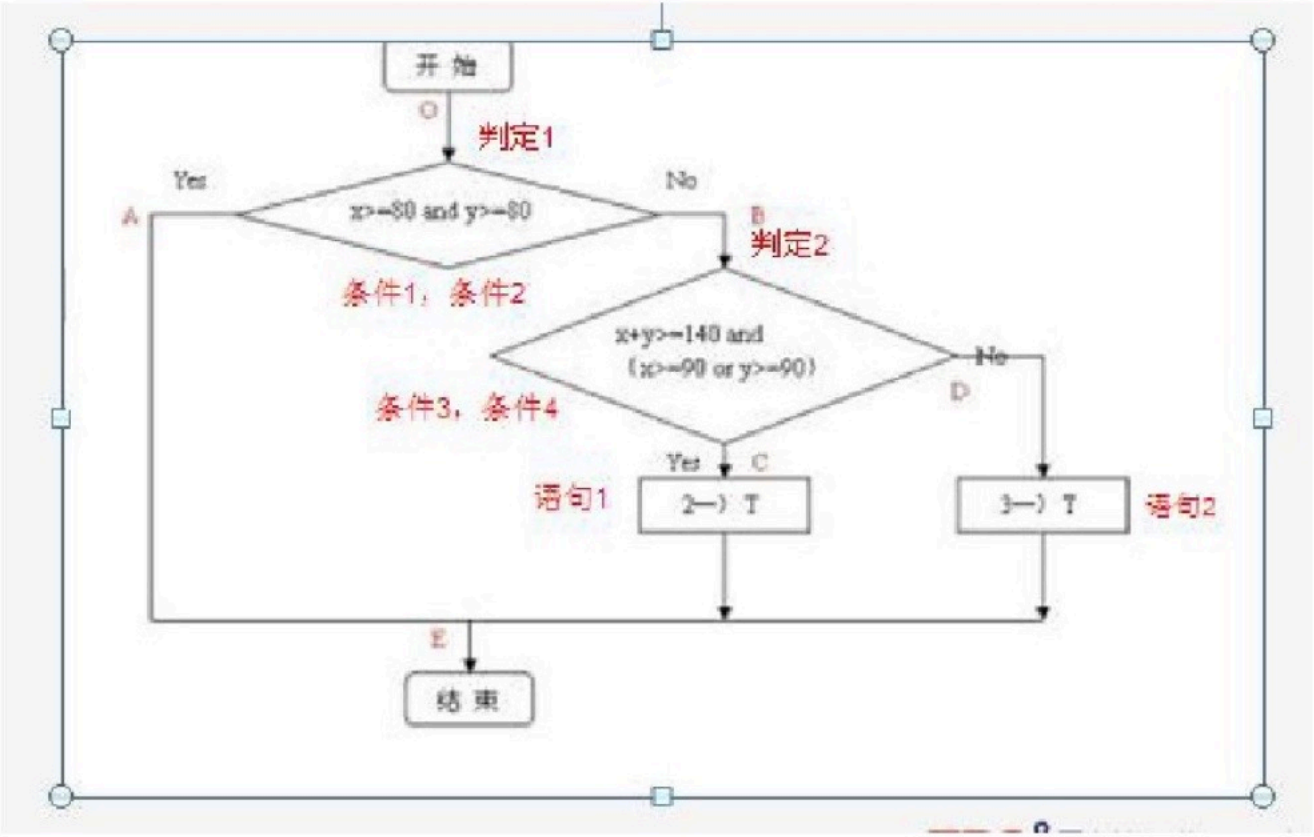
L4(a c d)=(A>1)and(B=0)

and(A<>2)and(X<=1)



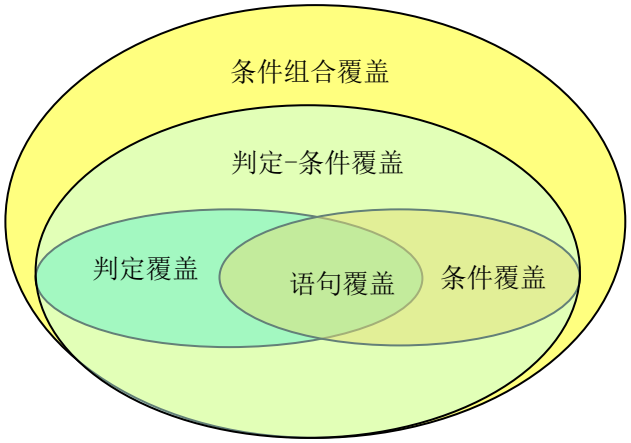
逻辑覆盖中术语：判定、条件

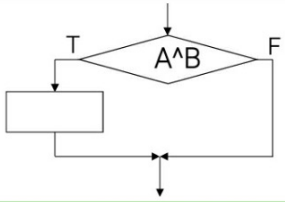
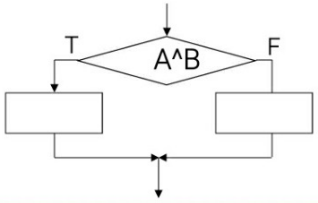
逻辑覆盖实例讲解

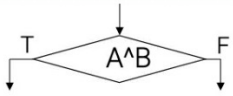
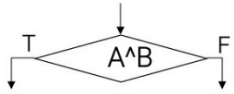
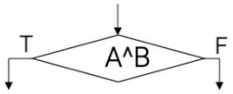


逻辑覆盖测试方法

逻辑覆盖测试是通过对程序逻辑结构的遍历实现程序的覆盖。从覆盖源代码的不同程度可以分为以下六个标准：语句覆盖、判定覆盖（又称为分支覆盖）、条件覆盖、判定-条件覆盖（又称为分支-条件覆盖）、条件组合覆盖和路径覆盖。



覆盖标准	程序结构举例	测试用例 应满足的条件
语句覆盖		A^B=.T.
判定覆盖		A^B=.T. A^B=.F.

覆盖标准	程序结构举例	测试用例应 满足的条件
条件覆盖		A=.T.　A=.F. B=.T.　B=.F.
判定/条件 覆盖		A^B=.T. , A^B=.F. A=.T.　A=.F. B=.T.　B=.F.
条件组合 覆盖		A=.T. ^ B=.T. A=.T. ^ B=.F. A=.F. ^ B=.T A=.F. ^ B=.F.

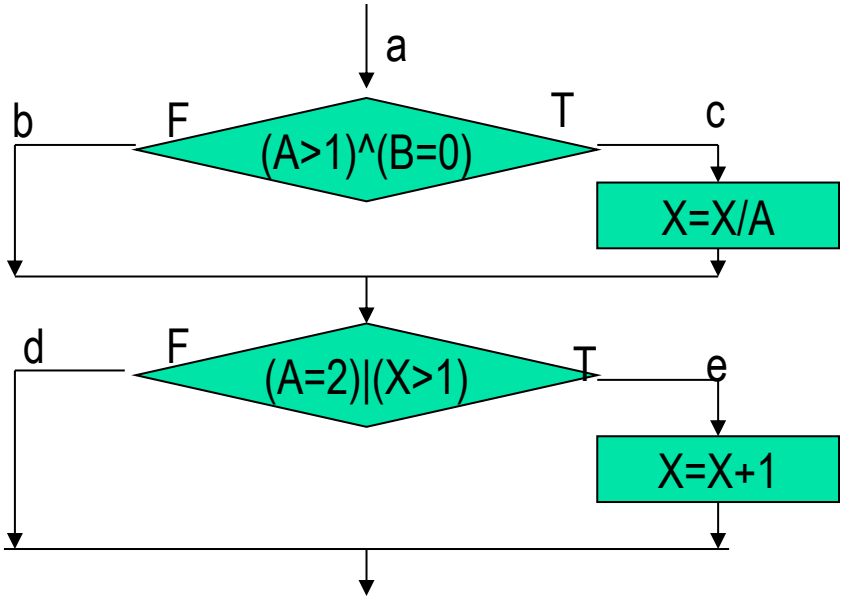
语句覆盖用例

语句覆盖就是设计若干个测试用例，运行所测程序，使得每一可执行语句至少执行一次。上例中可执行语句都在路径L1上，选择L1路径就可实现语句覆盖。

[（A，B，X），（A，B，X）]

例：[（2，0，4），（2，0，3）]

问题：如果运算符错误，则测不出来。



例：语句覆盖

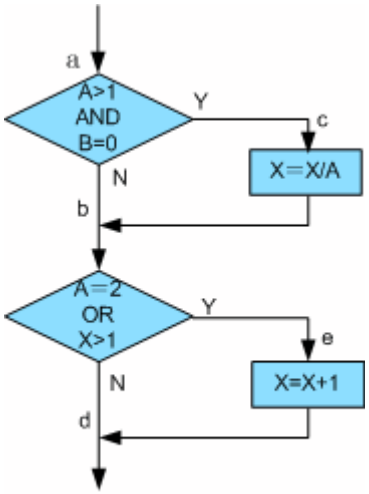


图6.4

设计1个设计用例，即可覆盖全部的语句。

判定覆盖用例

设计若干用例运行所测程序，使程序中每个判断的取真和取假分支至少经历一次。上例中选取L1和L2即可满足要求。

例：

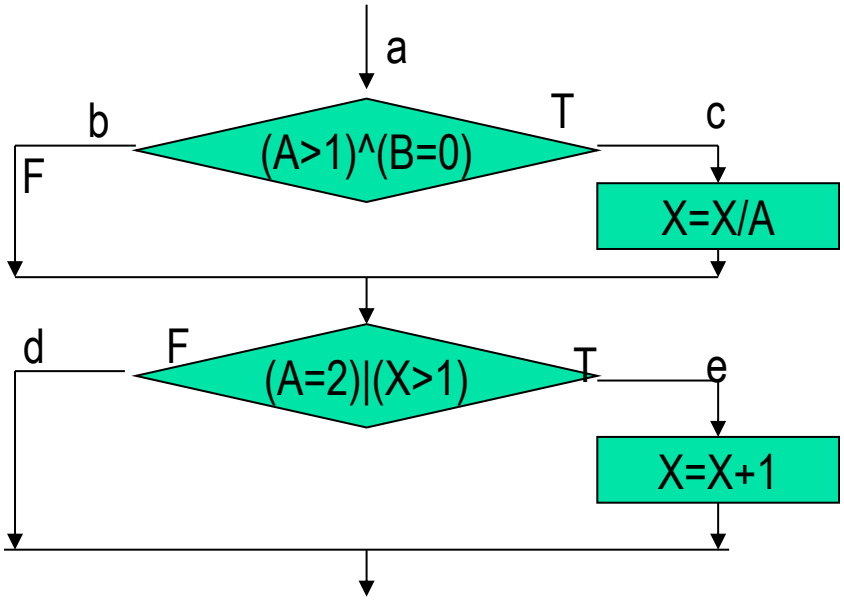
[（2，0，4），（2，0，3）] 覆盖（ace）

[（1，1，1），（1，1，1）] 覆盖（abd）

选择路径L3，L4还可得另一组测试用例。

[（2，1，1），（2，1，2）] 覆盖（abe）

[（3，0，3），（3，1，1）] 覆盖（acd）



问题：测不出某个变量的错误。

例：判定覆盖

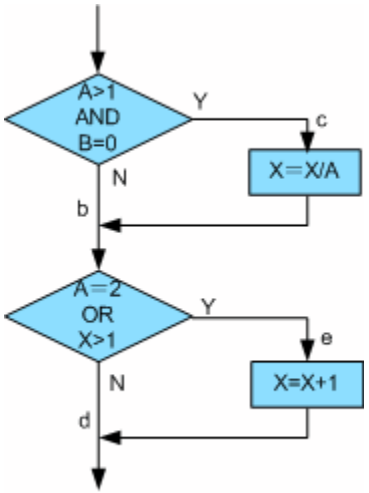


图6.4

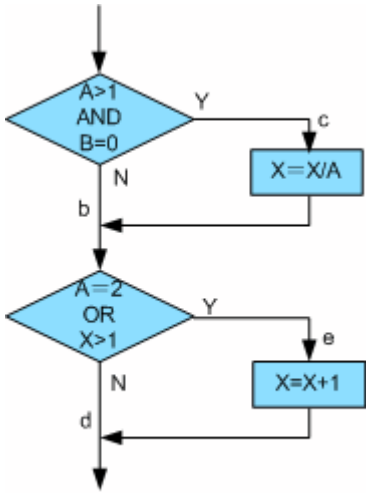


图6.4

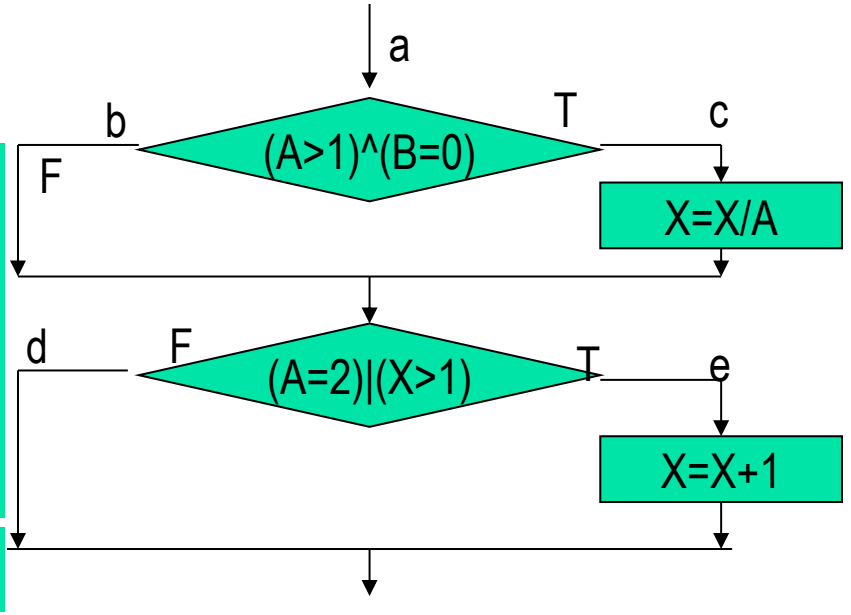
设计2个测试用例，运行所测程序，使程序中每个判断的取真和取假分支至少经历一次。

条件覆盖用例

设计若干用例，使程序中每个判断的每个条件的可能取值至少执行一次。

例：上例中对第一个判断有：A>1, B=0

对第二个判断有：A=2, X>1



用例设计：

[(2,0,4),(2,0,3)] a c e 或：[(1,0,3),(1,0,4)] a b e

[(1,0,1),(1,0,1)] a b d [(2,1,1),(2,1,2)] a b e

[(2,1,1),(2,1,2)] a b e

未考虑判定覆盖

例：条件覆盖

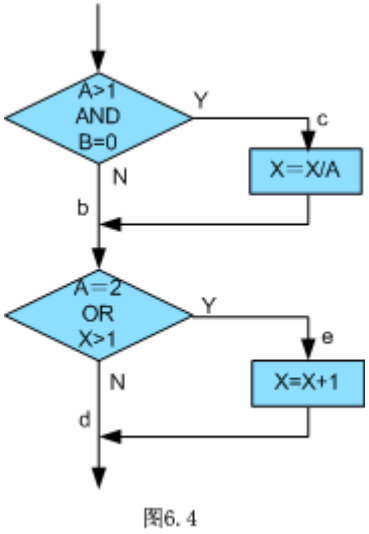


图6.4

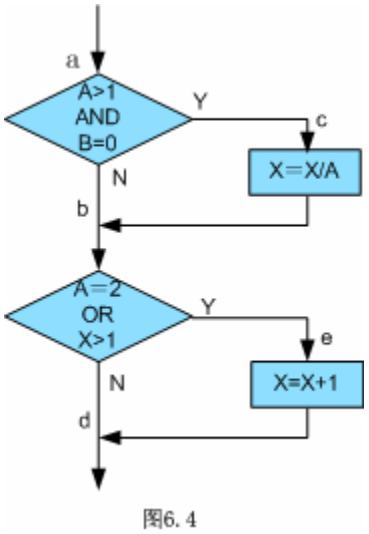


图6.4

设计2个测试用例，确保程序中每个判断的真、假取值至少执行一次。

判定-条件覆盖用例

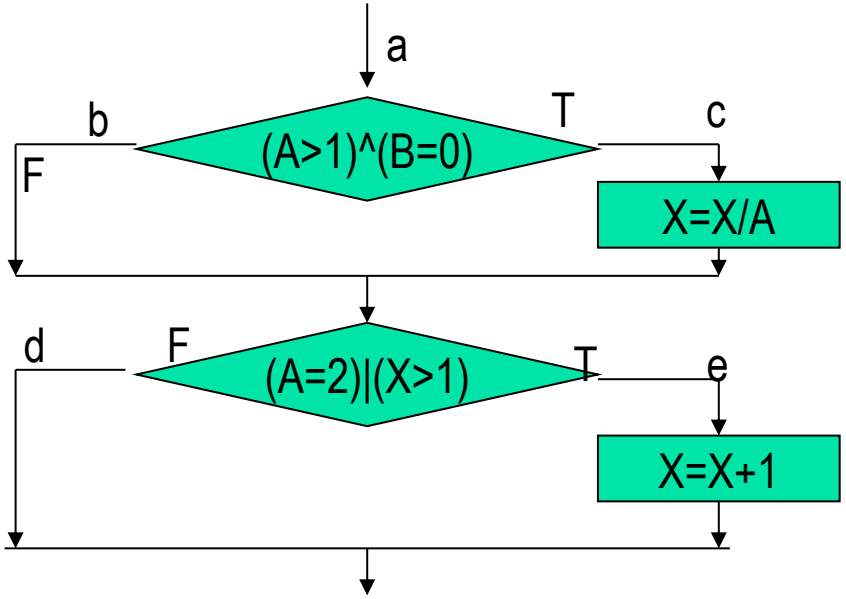
设计足够测试用例，使判断中每个条件的所有可能取值至少执行一次，同时每个判断的所有结果至少执行一次。

上例中取各值为真和假即可覆盖8个条件取值和4个判断分支。

例：

[（2，0，4），（2，0，3）] ace

[（1，1，1），（1，1，1）] abd



例：判定-条件覆盖

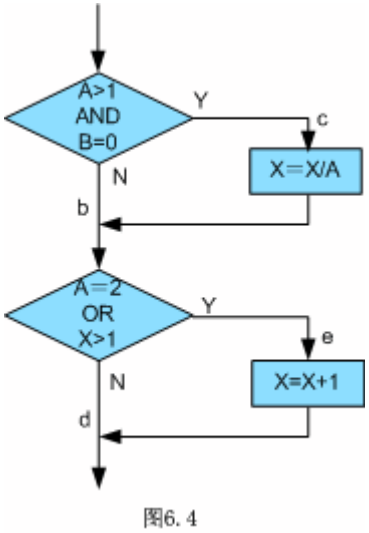


图6.4

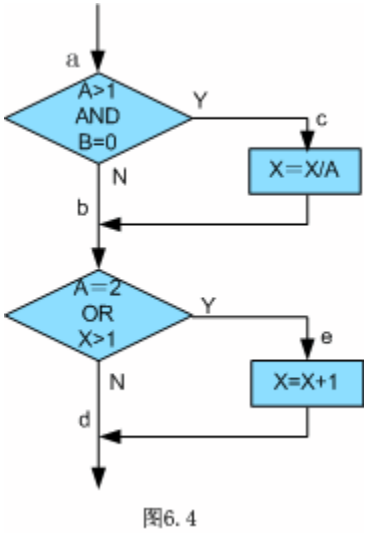


图6.4

设计2个测试用例，使判断中每个条件的所有可能取值至少执行一次，同时每个判断的所有结果至少执行一次。

条件组合覆盖用例

设计足够的测试用例，使每个判断的所有可能的条件取值组合至少执行一次。

上例中可能的组合为：

第一个判断：A>1, B=0　A>1, B<>0　A<=1, B=0　A<=1, B<>0

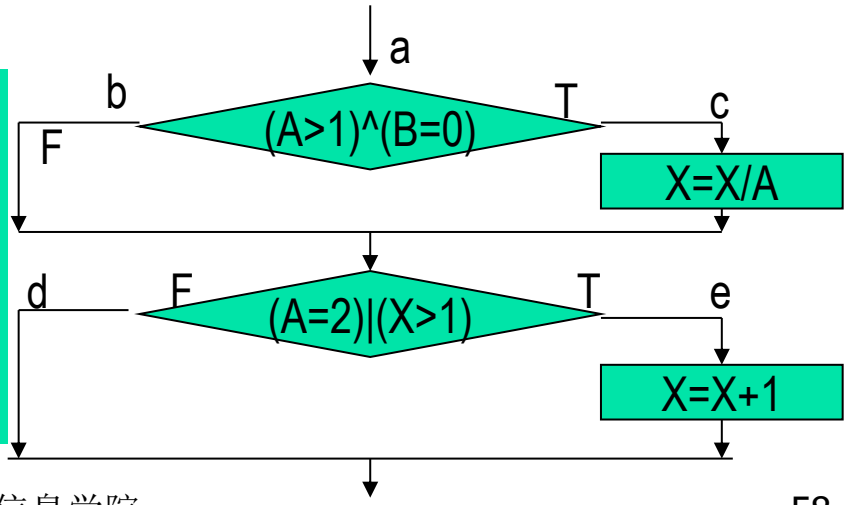
第二个判断：A=2, X>1　A=2, X<=1　A<>2, X>1　A<>2, X<=1

测试用例：[（2，0，4），（2，0，3）] ace

[（2，1，1），（2，1，2）] abe

[（1，0，3），（1，0，4）] abe

[（1，1，1），（1，1，1）] abd



例：条件组合覆盖

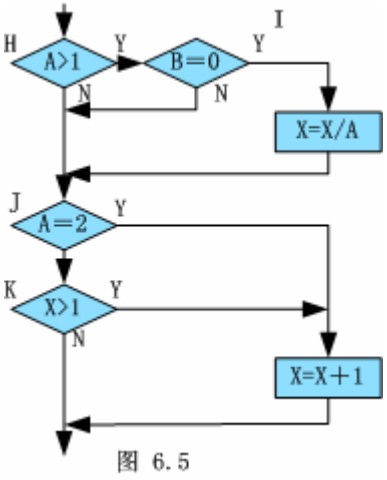


图 6.5

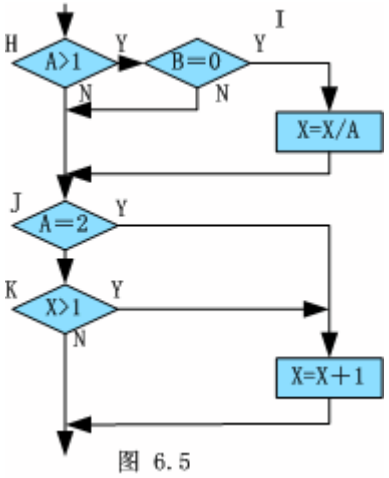


图 6.5

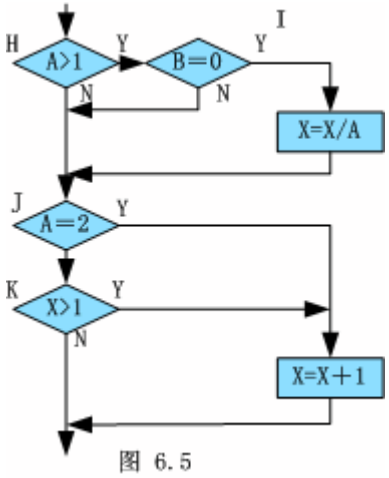


图 6.5

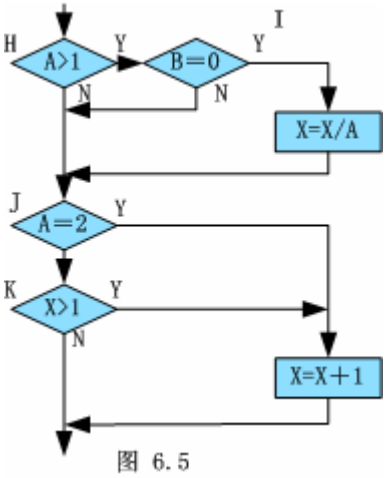


图 6.5

设计4个测试用例，使每个判断的所有可能的条件取值组合至少执行一次。

路径测试用例

设计足够的测试用例，覆盖程序中所有可能的路径。

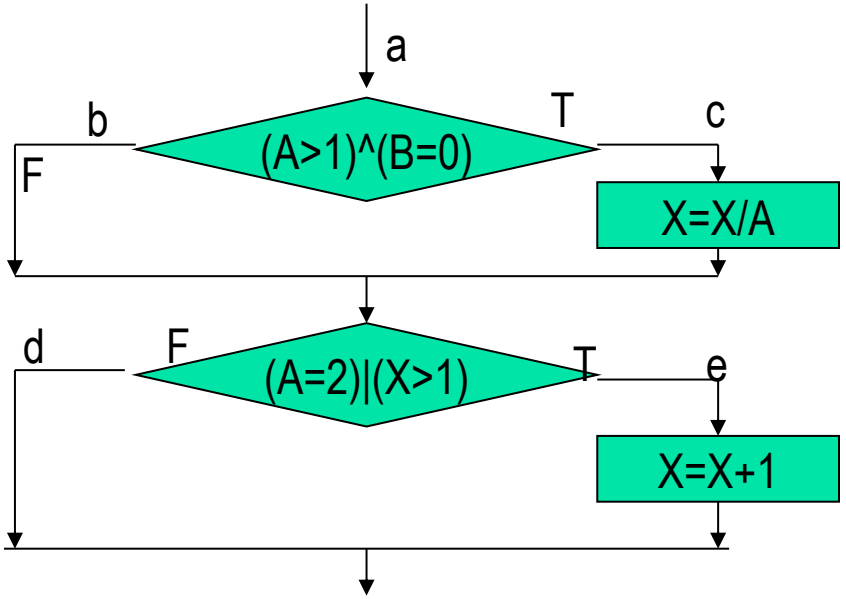
用例：

[（2，0，4），（2，0，3）] ace

[（1，1，1），（0，1，1）] abd

[（1，1，2），（1，1，3）] abe

[（3，0，3），（3，0，1）] acd



基本路径测试方法

实际程序中路径是个很庞大的数字，所有路径覆盖是不现实的，测试中把路径数压缩在一定范围内，称为**基本路径测试**。

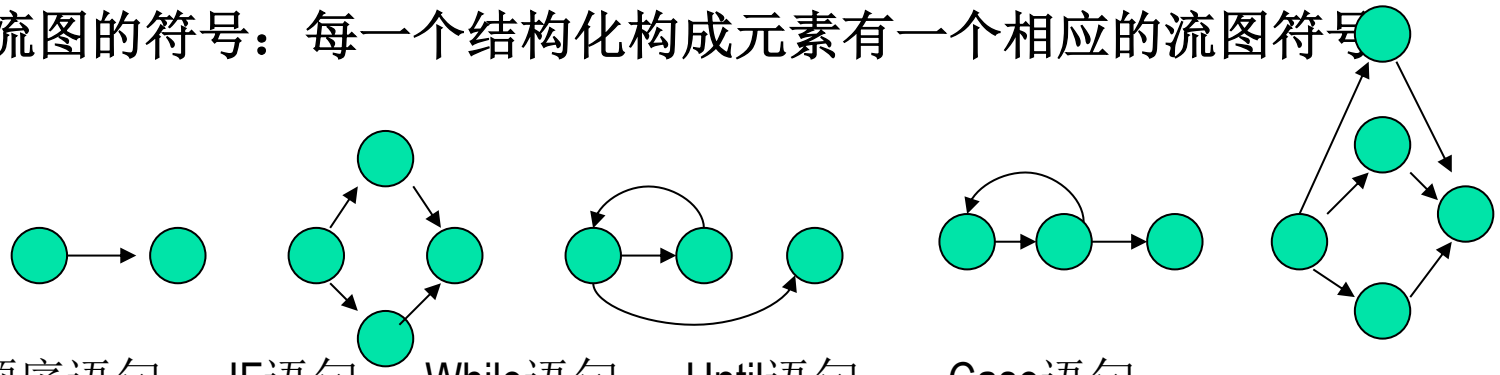
它是在程序控制流图的基础上，通过分析控制构造的环路复杂性测度，导出基本可执行路径的基本集合，从而设计测试用例。该基本集导出的测试用例保证程序的每一个可执行语句至少执行一次。

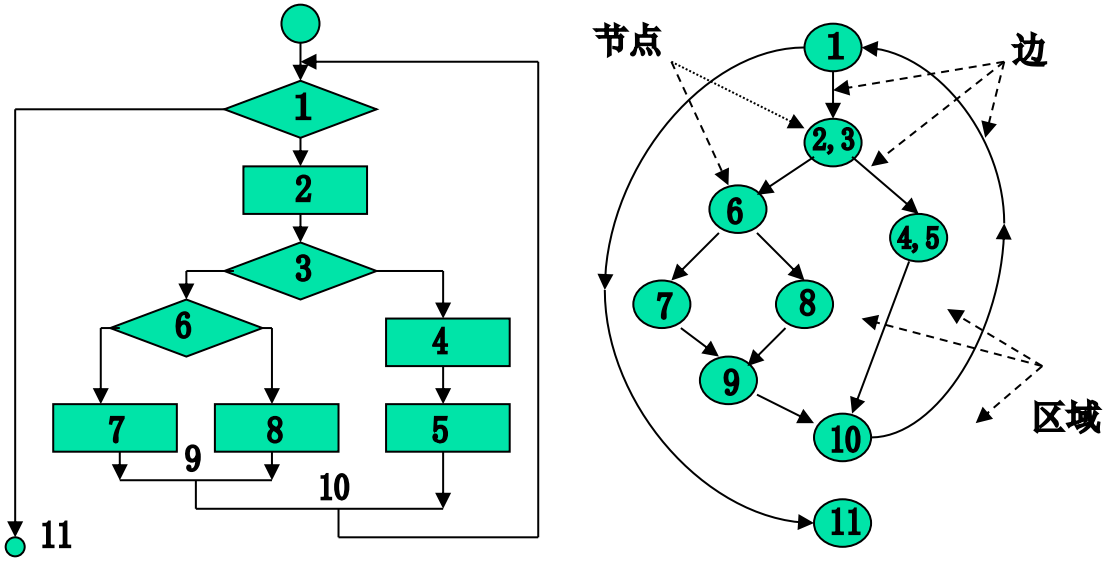
基本路径测试用例设计（**解决循环问题，计算复杂度**）：

- **程序的控制流图**
- **程序环路复杂性**
- **导出测试用例**
- **图矩阵**

控制流图

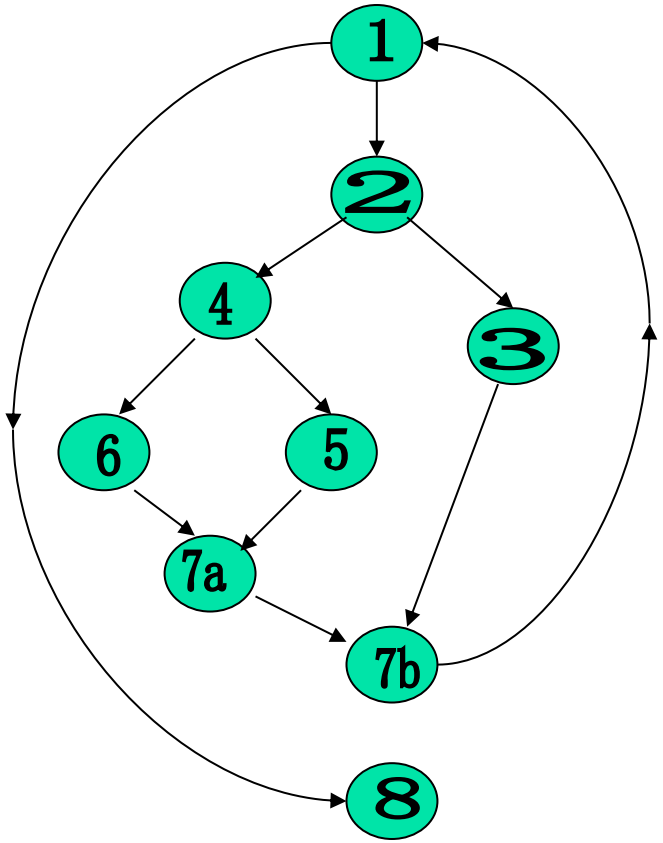
流图的符号：每一个结构化构成元素有一个相应的流图符号





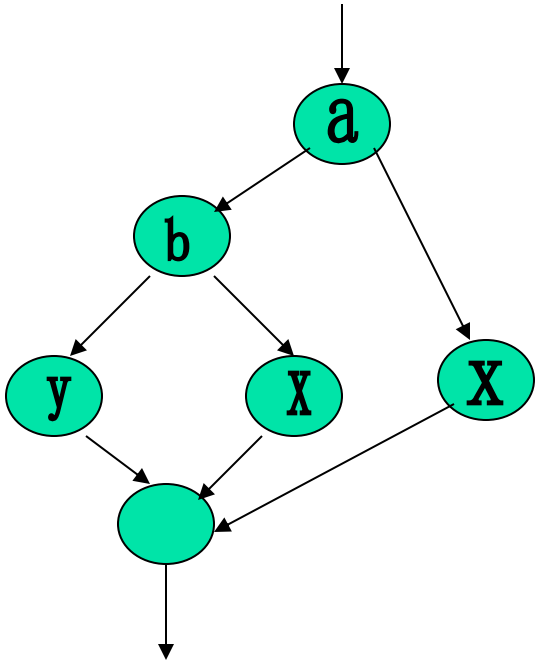
(flow chart到flow graph)

PDL翻译成流图




```
PDL
    Procedure: sort
 1:     do while records remain
            read record;
 2:         if record field 1 = 0
 3:             than process record;
                    store in buffer;
                    increment counter;
 4:             else if record field 2 = 0
 5:                 than reset counter;
 6:                 else process record;
                    store in file;
7a:             end if
            end if
7b:     end do
 8:end
```

复合逻辑的流图




```
……
IF a OR b
    THEN procedure x
    ELSE procedure y
ENDIF
……
```

转化成简单分支形式

程序环路复杂性

环路复杂性是一种为程序逻辑复杂性提供定量测度的软件度量，计算所得的值定义了程序基本集的独立路径数量，提供了确保所有语句至少执行一次的测试数量上界。

独立路径是指程序中至少引进一个新的处理语句集合或一个新条件的任意路径，即至少包括一条在定义路径前没有用到的边。

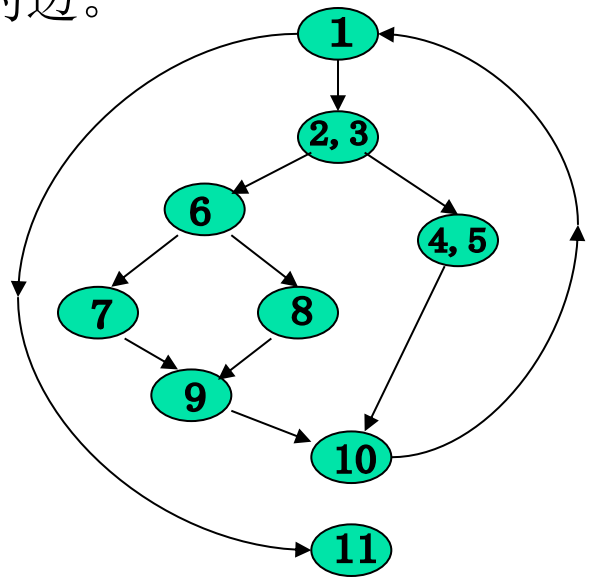
上例中独立路径为：

路径一：1-11

路径二：1-2-3-4-5-10-1-11

路径三：1-2-3-6-8-9-10-1-11

路径四：1-2-3-6-7-9-10-1-11



环路复杂性计算

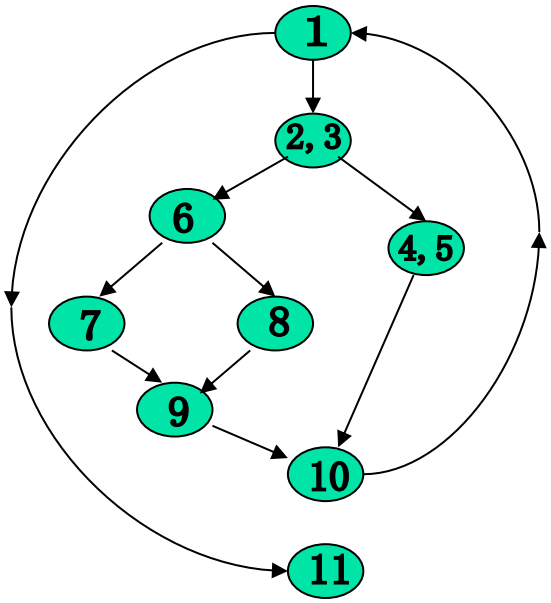
环路复杂性计算以图论为基础可用三种方法计算

- 流图中区域数量对应与环路的复杂性
- 给定流图G的环路复杂性计算：

V(G)=E-N+2　E边的数量，N节点数

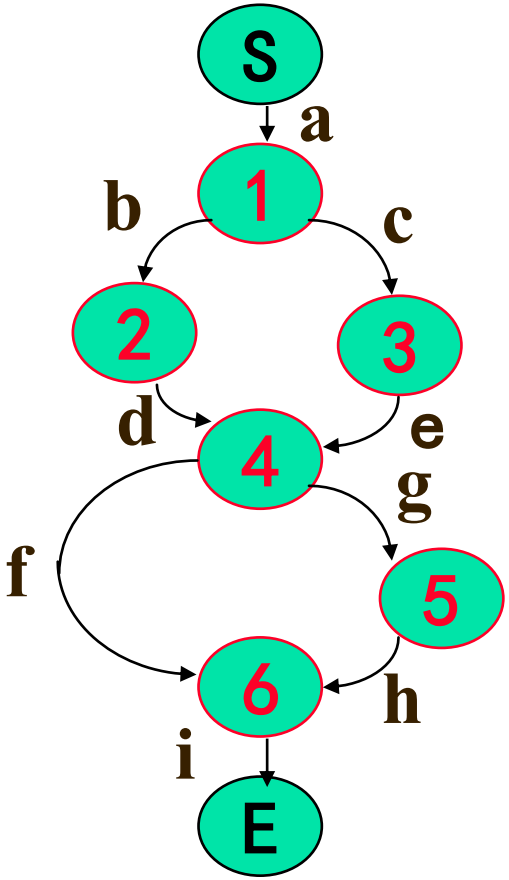
- 给定流图G的环路复杂性计算：

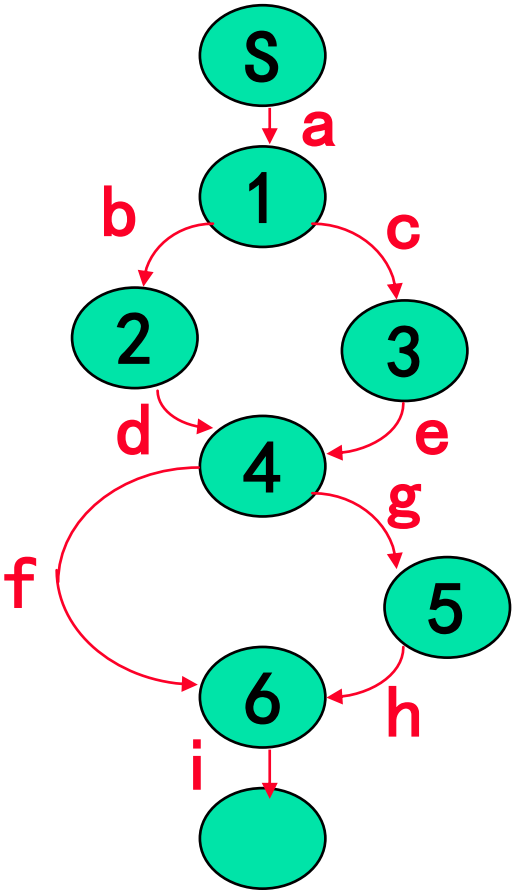
V(G)=P+1　P是流图G中判定节点的数量



1. 流图中有四个区域
2. V(G)=11条边-9个节点+2=4
3. V(G)=3个判定节点+1=4

点覆盖和边覆盖



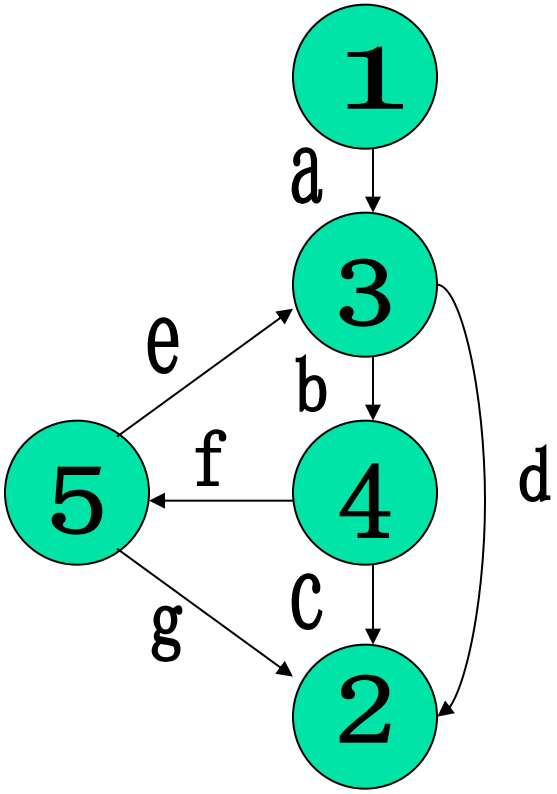


导出测试用例

- 从PDL导出流图
- 确定流图的复杂性
- 确定线性独立的路径的一个基本集
- 准备测试用例

图矩阵

用图矩阵实现确定基本测试路径自动化

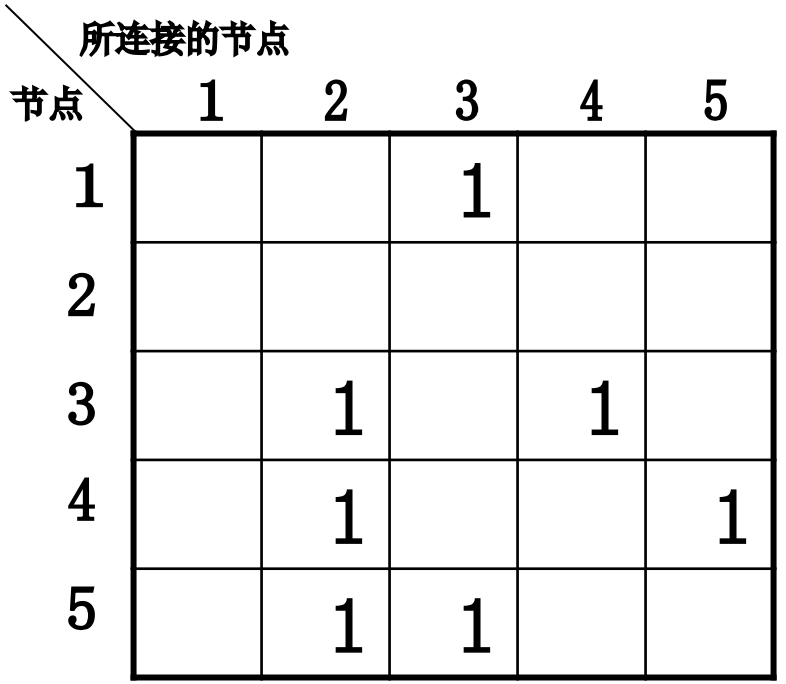


节点 \ 所连接的节点	1	2	3	4	5
1			a		
2					
3		d		b	
4		c			f
5		h	e		

图矩阵例

最简单的情况下，连接权值是1或0，但是，连接权值可以赋予其它属性：

- 执行连接边的概率
- 穿越连接的处理时间
- 穿越连接时所需内存
- 穿越连接时所需资源



计算：

1-1=0

2-1=1

2-1=1

2-1=1

3+1=4

McCabe度量法

- 它是一种基于程序控制流的复杂性度量方法，定义的度量值又称环路复杂度。
- 它是基于一个程序模块的程序图中环路的个数。如果把程序流程图中每个处理符号都退化成一个结点，原来联结不同处理符号的流线变成连接不同结点的有向弧，这样得到的有向图就叫做程序图。
- 计算有向图*G*的环路复杂性的公式：

$$V(G)=m-n+2$$

其中，*V*（*G*）是有向图*G*中的环路个数，*m*是图*G*中有向弧个数，*n*是图*G*中结点个数。

- *McCabe*环路复杂度度量值实际上是为软件测试的难易程度提供了一个定量度量的方法，同时也间接地表示了软件的可靠性。
- 实验表明，源程序中存在的错误数以及为了诊断和纠正这些错误所需的时间与*McCabe*环路复杂度度量值有明显的关系。

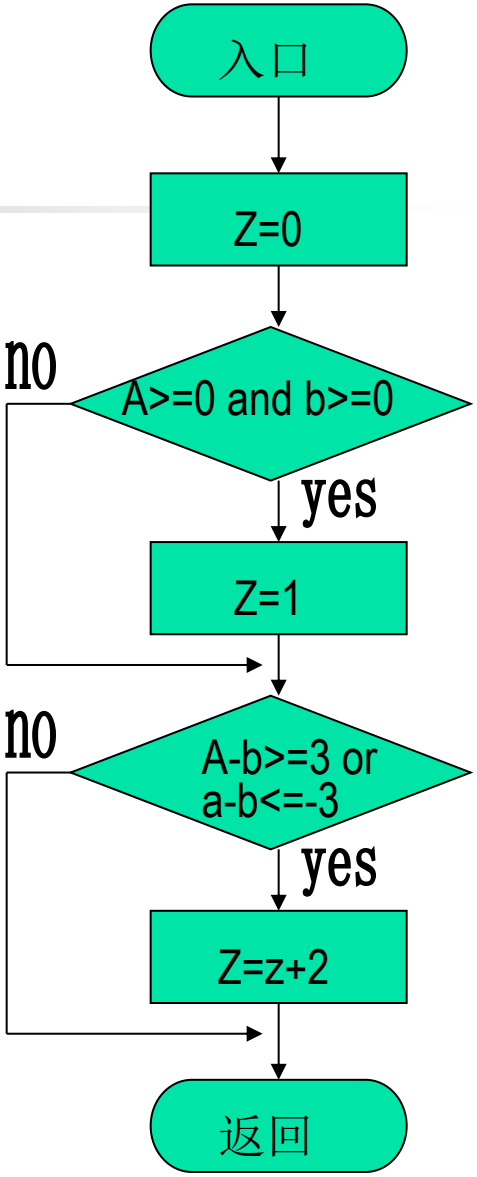
利用McCabe环路复杂度度量的几点说明：

- 环路复杂度取决于程序控制结构的复杂度。当程序的分支数目或循环数目增加时其复杂度也增加 。环路复杂度与程序中覆盖的路径条数有关。
- 环路复杂度是可加的。例如，模块A的复杂度为3，模块B的复杂度为4，则模块A与模块B的复杂度是7。
- McCabe建议，对于复杂度超过10的程序，应分成几个小程序，以减少程序中的错误 。Walsh用实例证实了这个建议的正确性。他发现，在McCabe复杂度为10的附近，存在出错率的间断跃变。
- McCabe环路复杂度隐含的前提是：错误与程序的判定加上例行子程序的调用数目成正比。而加工复杂性、数据结构、录入与打乱输入卡片的错误可以忽略不计。

课堂练习题

本流程描述了某子程序的处理流程，现要求用白盒测试法对子程序进行测试。

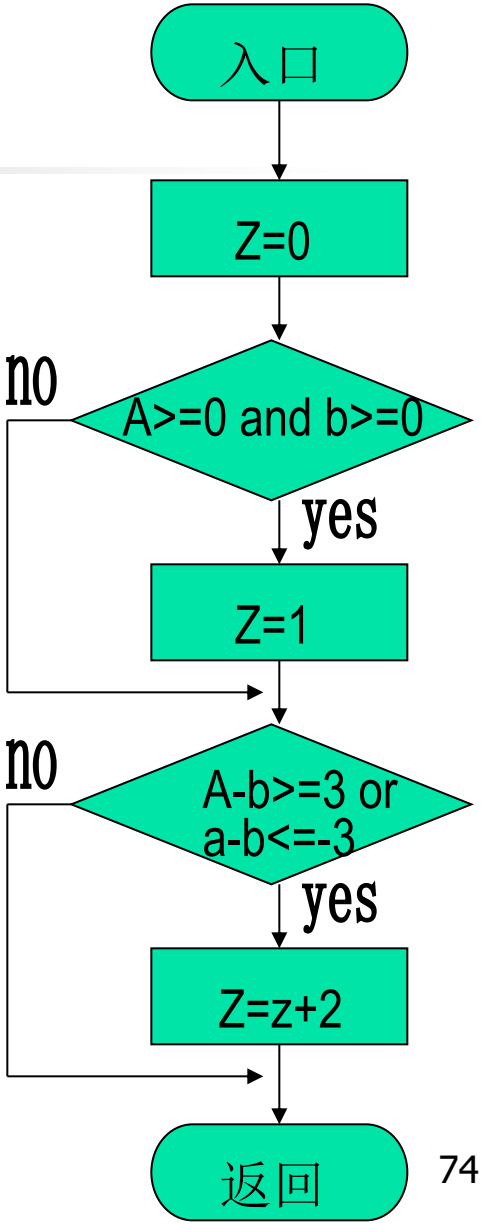
问题：根据判定覆盖、条件覆盖、判定条件覆盖、条件组合覆盖、路径覆盖等五种覆盖标准，从供选择的答案中分别找出满足相应覆盖标准的最小的测试数据组。



习题答案：④，③，⑥，⑧，⑦

课堂练习题续

①a=5, b=1
②a=5, b=-1
③a=5, b=1; a=-5, b=-1
④a=5, b=1; a=0, b=-1
⑤a=5, b=-1; a=-5, b=1; a=-5, b=-1
⑥a=5, b=1; a=0, b=0; a=-5, b=-1
⑦a=5, b=1; a=0, b=1; a=0, b=-1; a=-5, b=1
⑧a=5, b=1; a=0, b=-1; a=-5, b=1; a=-5, b=-1
⑨a=5, b=1; a=0, b=-1; a=0, b=1; a=-5, b=1; a=-5, b=-1
⑩a=5, b=1; a=5, b=0; a=5, b=-1; a=0, b=1; a=0, b=0;
a=0, b=-1; a=-5, b=1; a=-5, b=0; a=-5, b=-1



控制结构测试

基本路径测试简单高效但不够充分，人们提出一些变种的测试方法：

- 条件测试
- 数据流测试
- 循环测试
  - 简单循环
  - 嵌套循环
  - 串接循环
  - 不规则循环

条件测试

- 条件测试是检查程序模块中所包含逻辑条件的测试用例设计方法。
- 关系表达式的形式：

  E1 <relation-operator> E2

- 其中：
  - *E1*和*E2*是算术表达式。
  - <relation-operator>是下列之一：＜，≤，≠，（¬ ＝ ），＞，≥
  - 布尔运算符 OR |，AND &，NOT ¬

- 如果条件不正确，则至少有一个条件成份不正确，条件错误类型如下：
  - 布尔操作符错误（遗漏布尔操作符，布尔操作符多余或布尔操作符不正确）
  - 布尔变量错误
  - 布尔括号错误
  - 关系运算符错误
  - 算法表达式错误

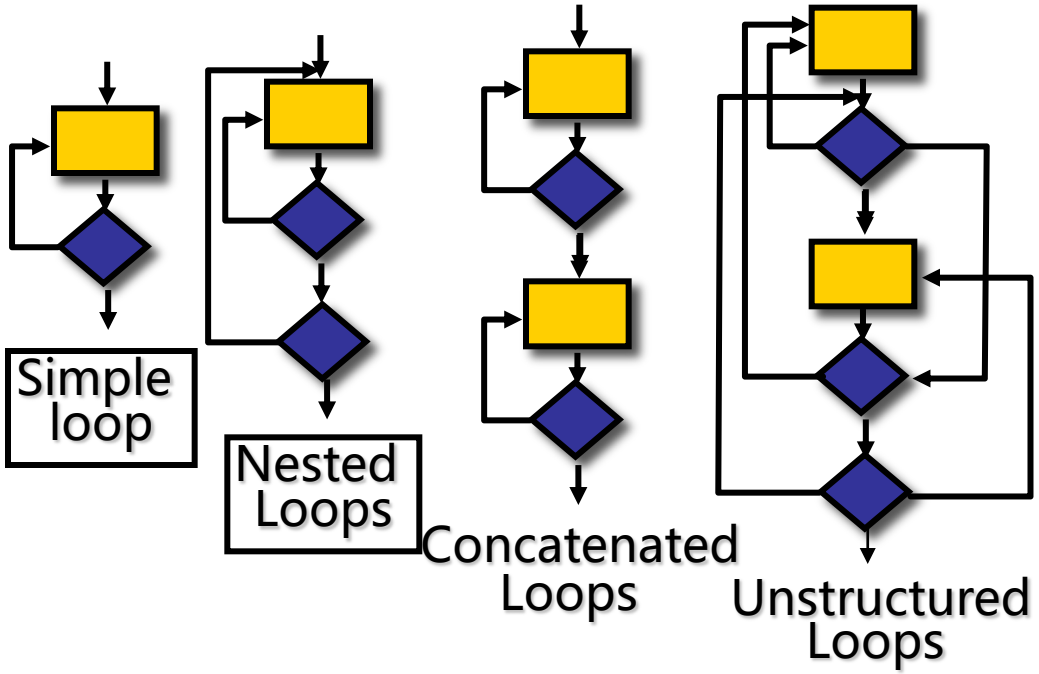
数据流测试

- 数据流测试按照程序中的变量定义和使用的位置来选择程序的测试路径。
- 测试策略是要求覆盖每个DU（*definition-use*）链至少一次——DU测试策略。
- 已经证明DU测试不能保证覆盖程序的所有分支，但是，不覆盖某个分支仅仅在于如下情况：*if-then-else*中的*then*没有定义变量，而且不存在*else*部分。
- 假设程序的每条语句赋予了唯一的语句编号，每个函数都不改变参数和全局变量。对于语句号为S的语句，

  $DEF(S)=\{X \mid \text{statement S contains a definition of } X\}$

  $USE(S)=\{X \mid \text{statement S contains a use of } X\}$
- 定义X的DU链式如[X, S, S′]，其中S和S′是语句号，X在DEF(S)和USE(S)中，而且语句S定义的X语句S′有效。

循环测试



简单循环（n是允许通过循环的最大次数）：

① 跳过整个循环

② 只有一次通过循环

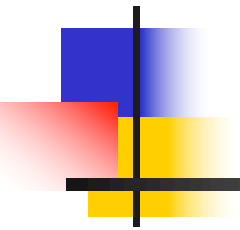
③ 两次通过循环

④ m次通过循环，其中m<n

⑤ n-1，n，n+1次通过循环

（续）

- 嵌套循环：如果要将简单循环的测试方法用于嵌套循环，可能的测试数就会随嵌套层数成几何级增加，减少测试数的方法：
  ① 从最内层循环开始，将其他循环设置为最小值。
  ② 对最内层循环使用简单循环测试，而使外层循环的迭代参数最小，而为范围外或排除的值增加其他测试。
  ③ 由内向外构造下一个循环的测试，但其他的外层循环为最小值，并使其他的嵌套循环为典型值。
  ④ 继续直到测试完所有的循环。
- 串接循环：如果串接循环的循环都彼此立，可以使用简单循环的策略测试，但是，如果循环不独立，则使用嵌套循环测试策略。
- 不规则循环：尽可能将这类循环重新设计为结构化的程序结构。



软件测试

6 黑盒测试技术



黑盒测试

- **黑盒测试**（black box testing），又称功能测试、数据驱动测试或基于规格说明的测试，在不考虑内部结构和内部特征，测试者只需要知道该程序输入和输出之间的关系或程序功能的情况下，依靠能够反映这一关系和程序功能需求规格的说明书，来确定测试用例和推断测试结果的正确性。
- 软件的黑盒测试一般是用来确认软件功能的正确性和可操作性。

黑盒测试的特点

黑盒测试定义：

在不考虑内部结构和内部特征，只需要知道程序输入和输出之间的关系或程序功能的情况下，依靠能够反映这一关系和程序功能需求规格的说明书，来确定测试用例和推断测试结果的正确性。

黑盒测试的原则

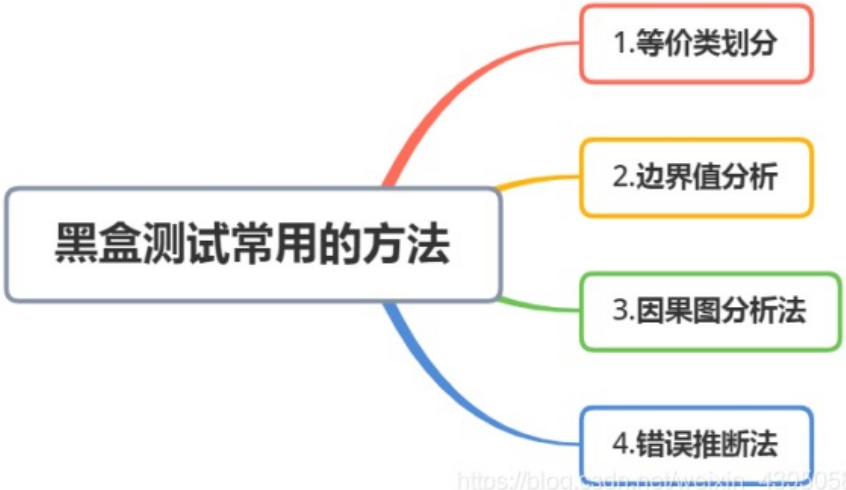
- 如何测试功能的有效性？
- 何种类型的输入会产生好的测试用例？
- 系统是否对特定的输入值尤其敏感？
- 如何分隔数据类的边界？
- 系统能够承受何种数据率和数据量？
- 特定类型的数据组合会对系统产生何种影响？

- 优点：
  - 适用面广，常用于功能测试、非功能测试及回归测试等；
  - 测试人员和编码人员相对独立，从用户的视角进行测试；
  - 有助于暴露任何与规格不一致或有歧义的问题；
  - 测试用例的设计可以不必等到编码完成，可以在规格完成之后马上进行。

- 缺点：
  - 测试数据来自规格说明书，测试结果的判定依赖于规格说明书的正确性；
  - 没有清晰、简明的规格，测试用例很难设计；
  - 只有一小部分可能的输入被测试到，穷举测试是不可能的。

黑盒测试用例

- 黑盒测试用于测试的后期，它不考虑控制结构，而是注重信息域。
- 设计黑盒测试用例要考虑：
  - 如何测试功能的有效性?
  - 何种类型的输入会产生好的测试用例?
  - 系统是否对特定的输入值尤其敏感?
  - 如何分隔数据类的边界?
  - 系统能够承受何种数据率和数据量?
  - 特定类型的数据组合会对系统产生何种影响?



等价划分

- 如果对象由具有对称性、传递性或自反性的关系连接，就存在等价类。
- 将程序的输入域划分为数据类，以便导出测试用例。
- 等价类划分试图定义一个测试用例，以发现同一等价类中的错误，从而减少开发测试用例数。

等价划分原则：

- 有效等价类；
- 无效等价类；
- 按照数值集合划分；
- 按照限制条件或规则划分；
- 细分等价类



如何划分等价类？

① 如果输入条件代表一个范围，可以定义一个有效等价类和两个无效等价类；

② 如果输入条件需要特定的值，可以定义一个有效等价类和两个无效等价类；

③ 如果输入条件代表集合的某个元素，可以定义一个有效等价类和一个无效等价类；

④ 如果输入条件是布尔式，可以定义一个有效等价类和一个无效等价类；

⑤ 如果规定输入条件必须遵守的规则，则可划分一个有效等价类(符合规则)和若干个无效等价类(从不同角度违反规则)；

⑥ 如已划分的等价类各元素在程序中的处理方式不同，则应将此等价类进一步划分成更小的等价类。

用等价类划分法设计测试用例步骤

① 形成等价类表，每一等价类规定一个唯一的编号；
② 设计一测试用例，使其尽可能多地覆盖尚未覆盖的有效等价类，重复这一步骤，直到所有有效等价类均被测试用例所覆盖；
③ 设计一新测试用例，使其只覆盖一个无效等价类，重复这一步骤直到所有无效等价类均被覆盖。

- 例：划分等价类法

输入条件	有效等价类	无效等价类
区号：三位数，但不是0和1开始	定义在200和999之间的数值	1）小于200 2）大于999

例1：用等价类划分法设计电话号码的测试用例。

某城市电话号码由三部分组成：

地区码：空白或3位数字
前　缀：非‘0’或‘1’开头的三位数字
后　缀：4位数字

第一步：电话号码等价类划分

输入条件	有效等价类	无效等价类
地区码	空白 (1) 3位数字 (2)	有非数字字符 (5) 少于3位数字 (6) 多于3位数字 (7)
前缀	从200到999之间的3位数字(3)	有非数字字符 (8) 起始位为‘0’ (9) 起始位为‘1’ (10) 少于3位数字 (11) 多于3位数字 (12)
后缀	4位数字(4)	有非数字字符 (13) 少于4位数字 (14) 多于4位数字 (15)

第二步：确定测试用例

对表中4个有效等价类可公用

下面两个测试用例：

测试数据	测试范围	期望结果
(　)276-2345	等价类(1)(3)(4)	有效
(635)805-9321	等价类(2)(3)(4)	有效

第三步：确定无效结果测试用例

对表中11个无效等价类应选择11个测试用例

测试数据	测试范围	期望结果
(20A)123-4567	无效等价类(5)	无效
(33)234-5678	无效等价类(6)	无效
(7777)345-6789	无效等价类(7)	无效
.	.	.
.	.	.
.	. (15)	无效

本例的15个等价类至少需要13个测试用例

例2:对招干考试系统“输入学生成绩”子模块设计测试用例

招干考试分三个专业，准考证号第一位为专业代号，如：

1-行政专业,
2-法律专业,
3-财经专业.

行政专业准考证号码为：110001~111215
法律专业准考证号码为：210001~212006
财经专业准考证号码为：310001~314015

例：准考证号码的等价类:

- 有效等价类:
  (1) 110001 ~ 111215
  (2) 210001 ~ 212006
  (3) 310001 ~ 314015
- 无效等价类:
  (4) -∞ ~ 110000
  (5) 111216 ~ 210000
  (6) 212007 ~ 31000
  (7) 314016 ~ +∞

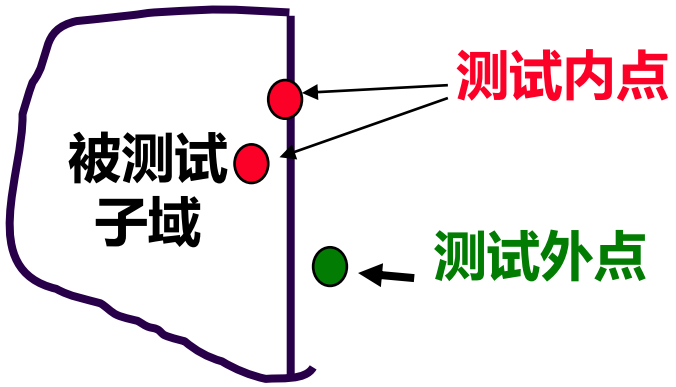
边界值分析

- 实践表明，输入域的边界比中间更容易发生错误，作为等价类划分法的补充边界值分析（*boundary value analysis, BVA*）可作为一种测试方法。

- 边界值分析法与等价类划分法区别：

  (1) 边界值分析不是从某等价类中随便挑一个作为代表，而是使这个等价类的每个边界都要作为测试条件。

  (2) 边界值分析不仅考虑输入条件，还要考虑输出空间产生的测试情况



边界值分析设计测试用例原则

1. 如输入条件代表以a和b为边界的范围，测试用例应包含a、b、略大于a和略小于b的值。
2. 如输入条件代表一组值，测试用例应当执行其中的最大值和最小值，还应测试略大于最大值和略小于最小值的值。

- 例：
  - 邮件收费规定 1～5 kg收费2元，则应对：

  0.9，1，5，5.1 kg 或 0.99，1，5，5.01 kg设计测试用例。
  - 一个输入文件可有1～255个记录则可分别设计有：

  1个、255个、0个、256个记录的输入文件

(续)

3. 如规格说明中提出输入输出的有序集（顺序文件、有序表等)，取有序集的第一个和最后一个元素做测试用例。
4. 如程序数据结构有预定的边界，应测试其边界的数据项。

- 例：程序中定义一数组，其元素下标的下界是0，上界是100，则应选择达到这个数组下标边界的值，如0与100作为测试用例。

例：取边界上下浮动值做测试用例

- 每日保险扣除额(输出项)在0～1165.25 元，则应设计测试用例使其恰好产生0元和1165.25元的结果， 此外还应考虑设计结果为负值或>1165.25元的测试用例，如：

  -0.01元和1165.26元。

- 由于输入值的边界不与输出值的边界相对应，所以要检查输出值的边界及产生超出输出值值域的结果不一定可能。

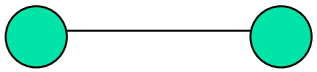
错误推测法

- 根据经验来设计测试用例的方法，例举出程序中可能有的错误和容易发生错误的特殊情况，根据它们选择测试方案。

因果图

- 因果图是根据分析时对各种条件可能产生的结果的一种对照分析法。
- 其测试用例的基本依据可以是分析阶段的决策表
  - 根据因果关系建立因果关系图
  - 建立决策表获得用例

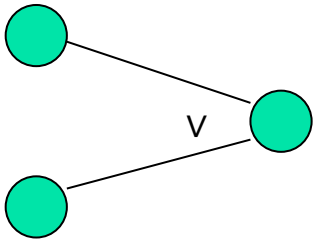
因果图图例：



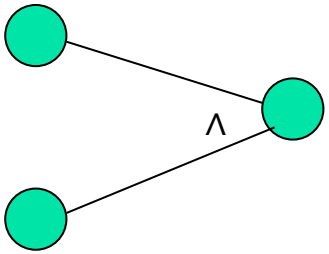
恒等



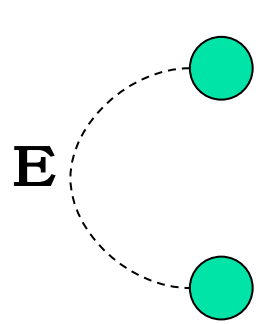
非

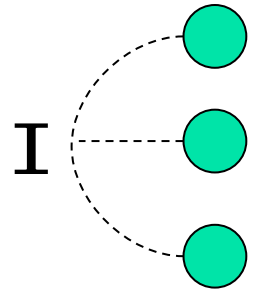


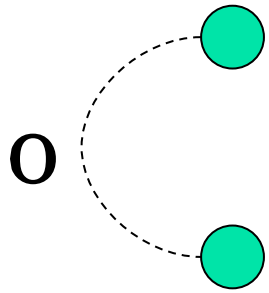
或

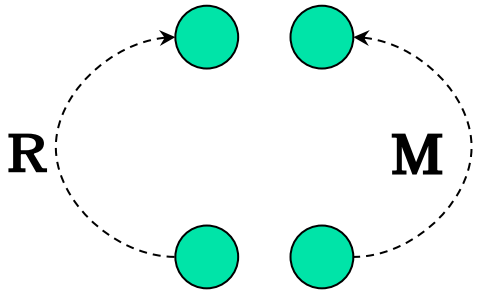


与









5种约束：互斥（E），包含（I），惟一（O），要求（R），屏蔽（M）

例1：自动饮料机软件的黑盒测试

- 问题：
  - 饮料单价1.5元。
  - 投入1.5元，按下“可乐”、“雪碧”或“红茶”按钮，测送出饮料。
  - 若投入2元，送出饮料时还退还5角。
- 解决方法：
  - 建立软件因果图
  - 设计测试用例

因果图例（续）

输入条件

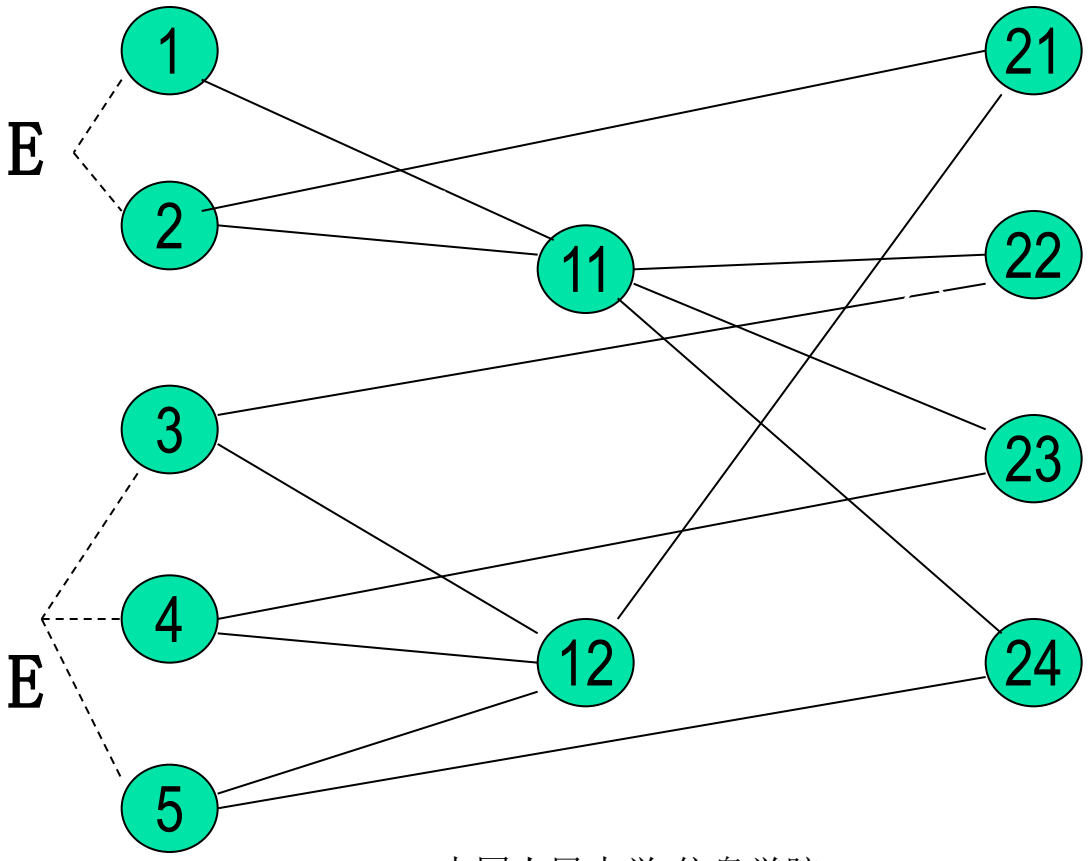
投1.5

投2元

按“可乐”

按“雪碧”

按“红茶”



输出条件：

退5角

送出“可乐”

送出“雪碧”

送出“红茶”

因果图例（续）

节点	操作	编号	1	2	3	4	5	6	7	8	9	10	11
输入节点	投1.5元	(1)	1	1	1	1	0	0	0	0	0	0	0
	投2元	(2)	0	0	0	0	1	1	1	1	0	0	0
	按“可乐”	(3)	1	0	0	0	1	0	0	0	1	0	0
	按“雪碧”	(4)	0	1	0	0	0	1	0	0	0	1	0
	按“红茶”	(5)	0	0	1	0	0	0	1	0	0	0	1
中间节点	已投币	(11)	1	1	1	1	1	1	1	1	0	0	0
	已按钮	(12)	1	1	1	0	1	1	1	0	1	1	1
输出	退5角	(21)	0	0	0	0	1	1	1	0	0	0	0
	送“可乐”	(22)	1	0	0	0	1	0	0	0	0	0	0
	送“雪碧”	(23)	0	1	0	0	0	1	0	0	0	0	0
	送“红茶”	(24)	0	0	1	0	0	0	1	0	0	0	0

课堂练习题

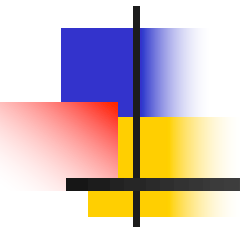
为求平方根的程序设计测试用例：

SQRT((x - 3) / (5 - x))

采用黑盒测试，要求用两种方法设计测试用例：

(1)等价划分法；

(2)边界值法。



软件测试

7 针对环境和应用的测试

针对环境和应用的测试

- GUI测试
- Client/Server体系结构测试
- 文档测试
- 实时系统
- Web测试

GUI测试

常见GUI测试指南：

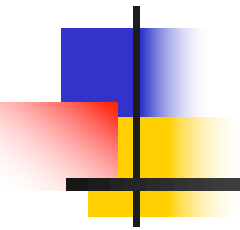
- 对于窗口
- 对于菜单和鼠标操作
- 对于数据项

C/S体系结构的测试

- 整体C/S测试策略(三个不同层次):
  - 客户端应以“分离的”模式被测试(不考虑服务器和底层网络的运行)
  - 客户端软件和关联的服务器端应用被一起测试(网络运行不被明显考虑)
  - 完整的C/S体系结构（包括网络运行和性能）被测试

C/S常用测试方法

- 客户端应用功能测试
- 服务器测试（协调和数据管理功能、性能）
- 数据库测试
- 事务测试
- 网络通信测试



软件测试

8 软件测试的策略

软件测试的策略

- 验证（verification）和确认（validation）
- 软件测试的组织
- 软件测试步骤：从下而上
- 测试完成的标准?

- 永远也不可能完成测试：开发者转向用户
- 时间不够或资金不够时
- Musa和Ackerman:1000小时CPU时间内不出错的概率大于0.995，则有95%信心说进行了足够的测试。
- 概率论模型$f(t)=(1/p)\ln(l_0pt+1)$，求导得故障率$l(t)$

测试结束的标准是什么？

1.测试目标达成
2.测试计划的实施
3.质量标准的达成
4.缺陷的修复
5.风险评估
6.客户满意度
7.法规和合规性
8.性能要求满足
9.文档和报告
10.决策和批准

总之，测试结束标准是一个综合考虑项目目标、质量、风险和客户需求的过程。

软件测试的策略

- 单元测试

  单元测试（unit testing），是指对软件中的最小可测试单元进行检查和验证。

- 集成测试

  集成测试（integration testing），也叫组装测试或联合测试。在单元测试的基础上，将所有模块按照设计要求（如根据结构图）组装成为子系统或系统，进行集成测试。

- 系统测试

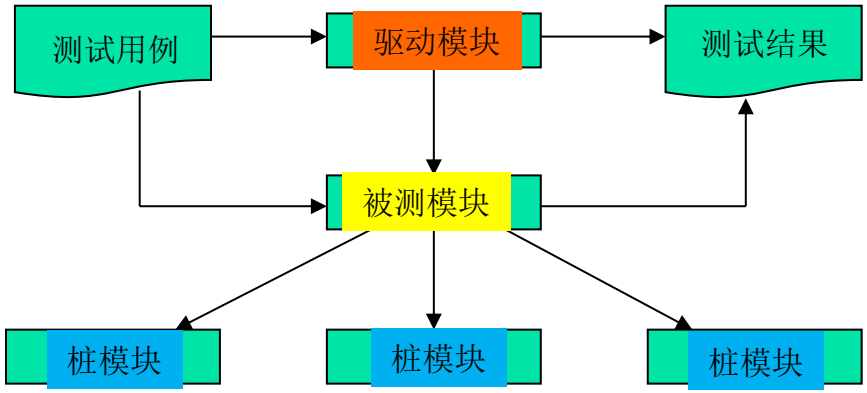
  系统测试（system testing），是对整个系统的测试，将硬件、软件、操作人员看作一个整体，检验它是否有不符合系统说明书的地方。

- 确认测试

  确认测试（confirmation testing），又称有效性测试，是对通过集成测试的软件在模拟的环境下进行的有效性测试，目的是要表明软件是可以工作的，并且符合”软件需求说明书”中规定的全部功能和性能要求。

单元测试

- 内容
  - 接口测试
  - 数据结构测试
  - 路径测试
  - 错误处理（非正常）测试
  - 边界测试
- 步骤
  - 驱动模块—辅助模块
  - 桩模块---辅助模块



驱动模块用来模拟被测模块的上层调用模块，功能要比真正的上级模块简单，仅仅是接受测试数据，并向被测模块传送数据，启动被测试模块，回收并输出测试结果。

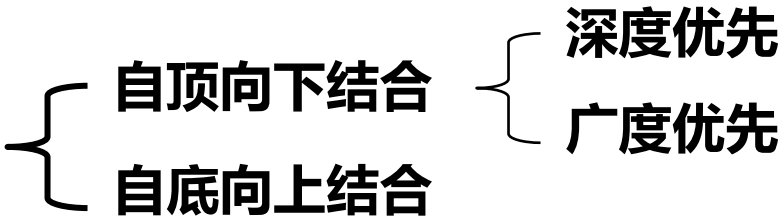
桩模块用来模拟被测模块在执行过程中所要调用的模块。它接受被测试输出的数据并完成它所指派的任务。

集成测试

- 模块连接时，穿越模块的数据是否会丢失；
- 是否会对另外模块产生不利影响；
- 各子模块组合能否达到预期的父模块功能；
- 全局数据结构是否有问题；
- 各模块误差的累积是否到了无法接受。
- 集成测试的策略
  - 一次性集成方式
  - 增殖式集成方式
  - 集成测试的组织和实施
  - 集成测试完成的标志

集成测试方法

- 通常采用黑盒测试技术
- 实施策略:
  - 非渐增式测试：一次就把所有通过了单元测试的模块组合在一起进行全程序的测试.
    - 缺点：发现错误难以诊断定位，又称“莽撞测试”。
  - 渐增式测试：从一个模块开始，测一次添加一个模块，边组装边测试，以发现与接口相联系的问题。



自顶向下集成

步骤：

(1) 主控模块为驱动模块，所有直属主模块的下属模块全用桩模块代替，测试主模块。

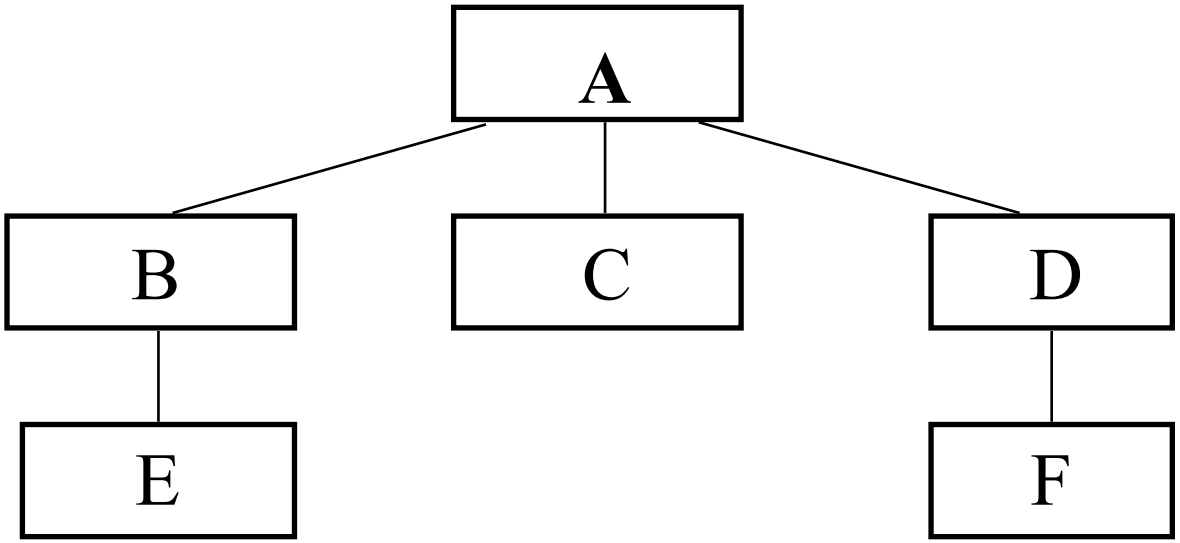
(2) 根据所选结合方法(先深度或先广度)， 每次用一实际模块替换相应桩模块。

(3) 模块结合一个，测试一个。

(4) 完成一组测试后，用实际模块替换另一个桩模块。

(5) 为保证不引入新错误，进行回归测试。

自顶向下集成方式

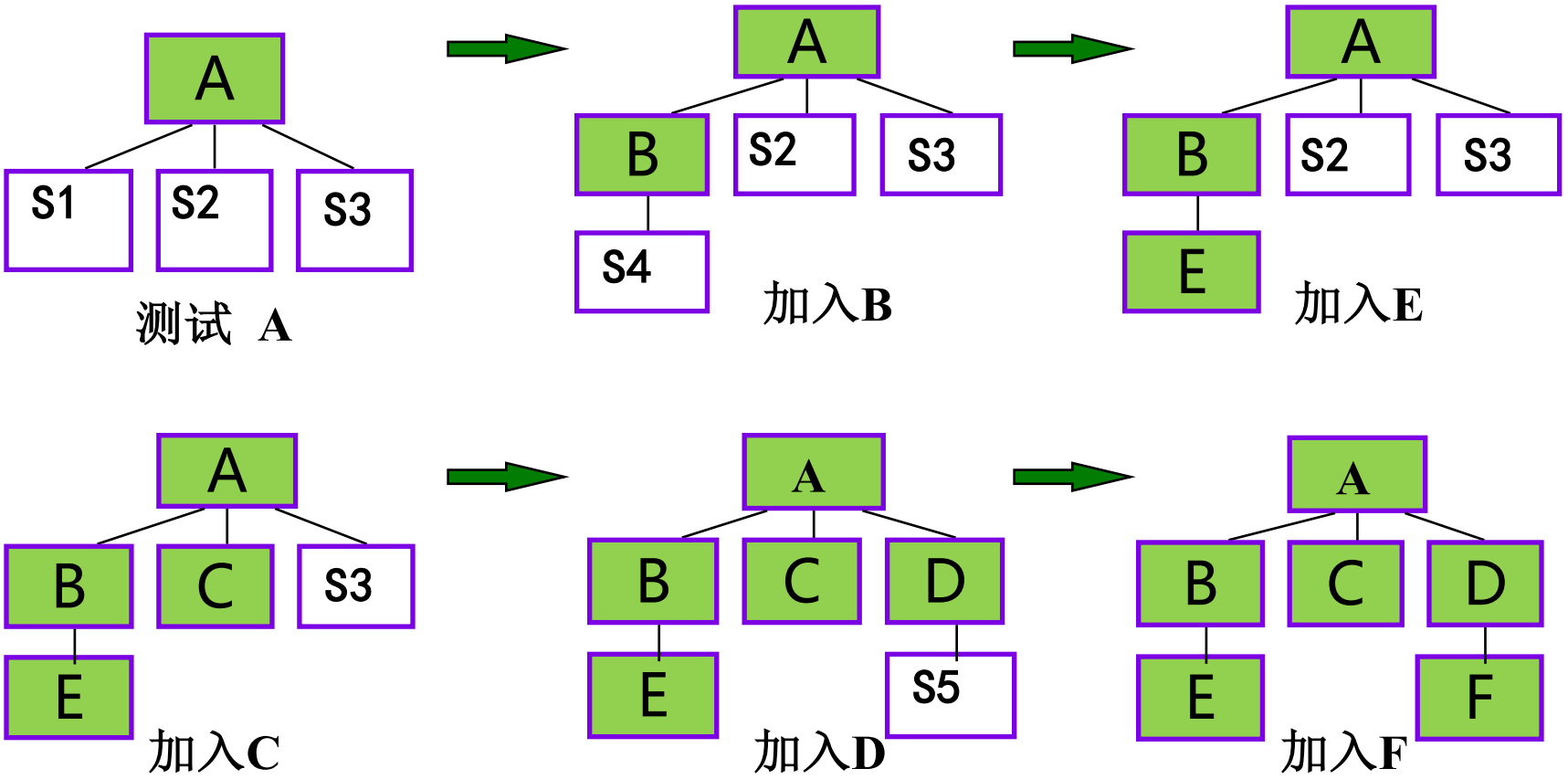


模块测试结合顺序

深度优先:A、B、E、C、D、F

广度优先:A、B、C、D、E、F

自顶向下集成方式(深度优先)



自顶向下集成的逻辑问题

- 测试初期，如上层模块对下层模块有很大的依赖性(要求低层模块返回的信息量大，种类多)，而桩模块替换低层模块后，不能向高层模块传送足够量的信息。
- 解决办法:
  - 把一些测试推迟到用实际模块替换桩模块后进行.
  - 使桩模块能模拟实际模块功能.
  - 自底向上集成测试.

自底向上集成

步骤:

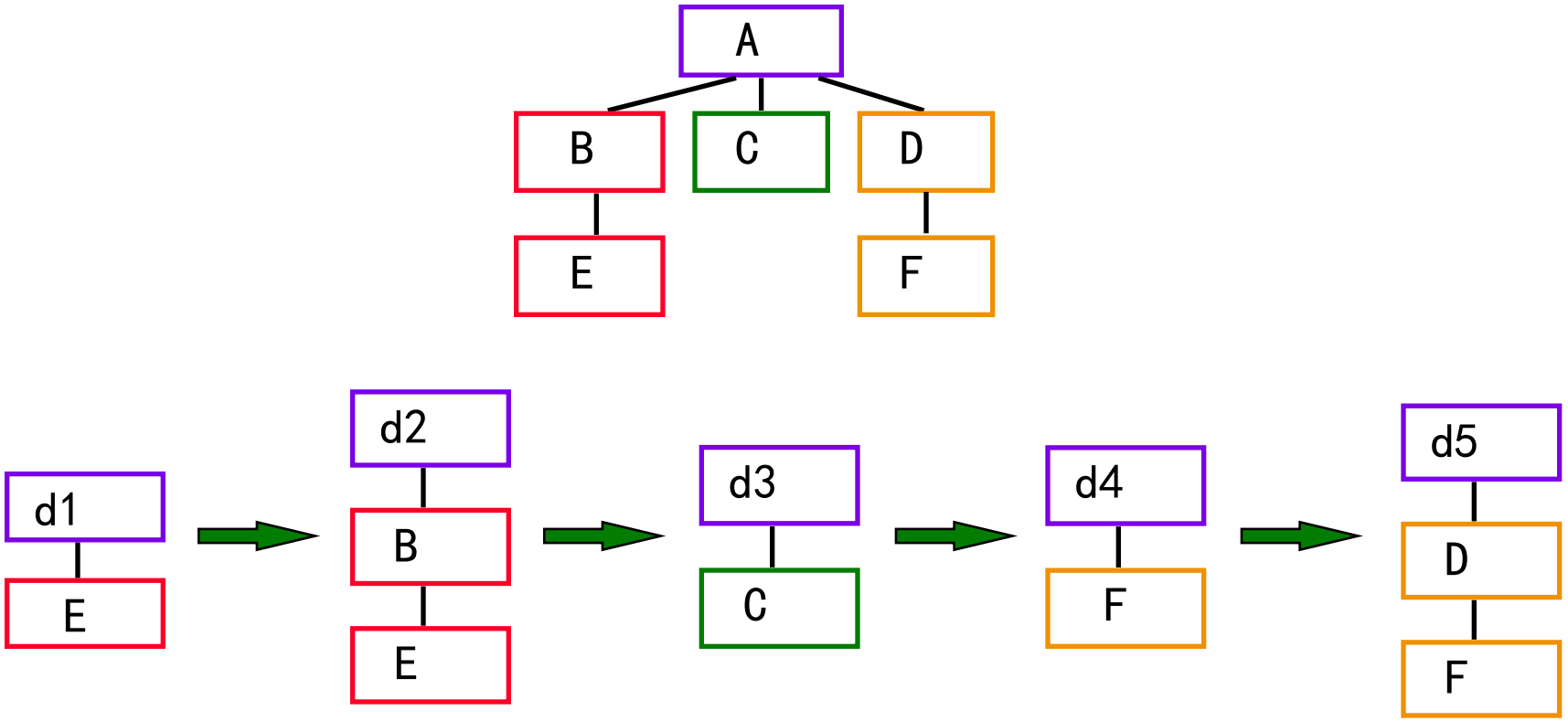
(1)对叶模块配以驱动模块对其测试，也可把最底层模块组合成实现某一特定软件功能的簇，由驱动模块对它测试。

(2)用实际模块代替驱动模块,与它已测试的直属模块组装成子系统。

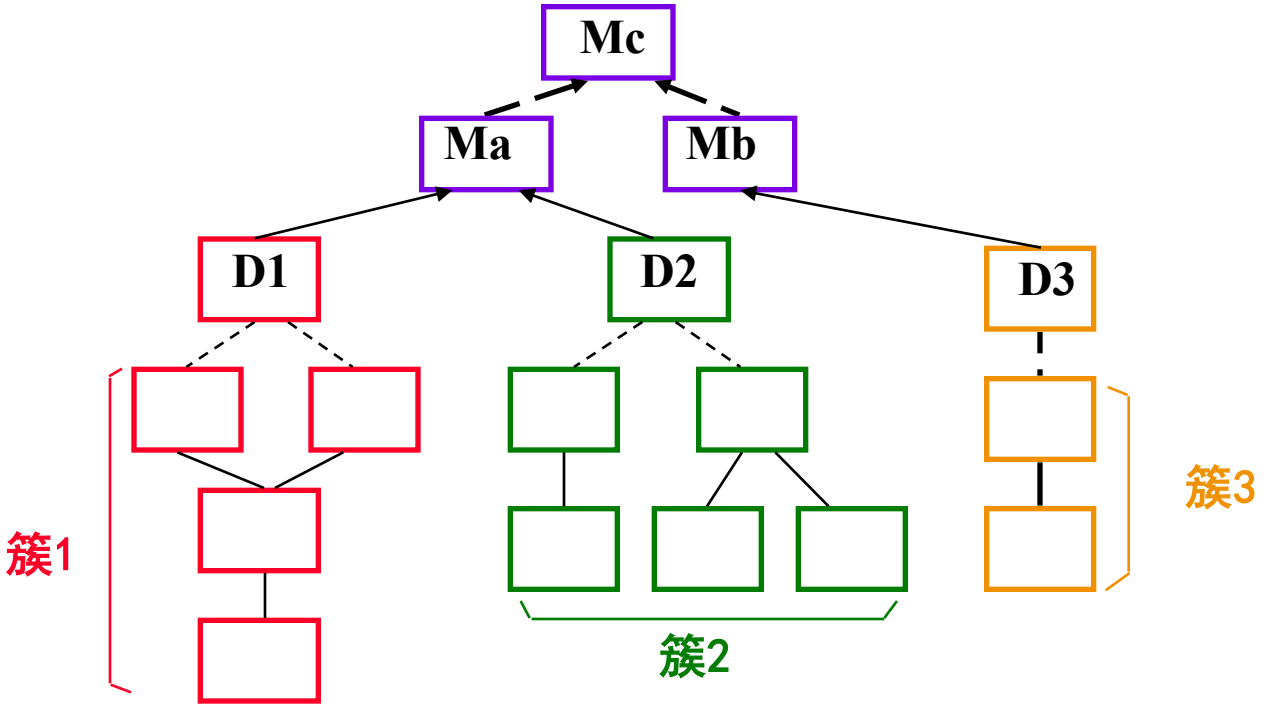
(3)为子系统配备驱动模块,进行新的测试。

(3)判断是否已组装到达主模块 是则结束，否则执行(2)

自底向上集成方式



自底向上集成方式



两种方法比较

- 自顶向下：可在测试早期实现并验证系统主要功能不需驱动模块，但需要桩模块。
- 自底向上：设计测试用例容易，但要到最后程序才能作为一个整体。
- 混合集成测试方法
  - 一般对软件结构的上层使用自顶向下结合的方法;
  - 对下层使用自底向上结合的方法。

系统测试

- 系统测试（***System Testing***）
  - 系统测试是将已经集成好的软件系统作为一个元素，与计算机硬件、外设、某些支持软件、数据和人员等其他元素结合在一起，在实际运行环境下进行的一系列测试。
- 系统测试方法
  - 功能测试、协议一致性测试
  - 性能测试、压力测试、容量测试、安全性测试、恢复测试
  - 备份测试、**GUI**测试、健壮性测试、兼容性测试、可用性测试
  - 安装测试、文档测试、在线帮助测试、数据转换测试

系统测试——功能测试（*Functional Testing*）

- 功能测试是系统测试中最基本的测试，它不管软件内部的实现逻辑，主要根据软件需求规格说明和测试需求列表，验证产品的功能实现是否符合需求规格。
- 功能测试主要发现以下错误：
  - 是否有不正确或遗漏的功能？
  - 功能实现是否满足用户需求和系统设计的隐藏需求？
  - 能否正确地接受输入？能否正确地输出结果？
- 常用的测试技术
  - 黑盒测试方法：等价类划分、边界值测试

系统测试——压力测试（*Press Testing*）

- 压力测试是检查系统在资源超负荷情况下的表现，特别是对系统的处理时间有什么影响。
- 压力测试的例子
  - 对于一个固定输入速率的单词处理响应时间
  - 在一个非常短的时间内引入超负荷的数据容量
  - 成千上万的用户在同一时间从网上登录到系统
  - 引入需要大量内存资源的操作
- 压力测试采用边界值和错误猜测方法，且需要工具的支持。

系统测试——安全性测试（*Security Testing*）

- 安全性测试检查系统对非法侵入的防范能力。
- 安全性测试期间，测试人员假扮非法入侵者，采用各种办法试图突破防线。
- 安全性测试的例子
  - 想方设法截取或破译口令
  - 专门定做软件破坏系统的保护机制
  - 故意导致系统失败，企图趁恢复之机非法进入
  - 试图通过浏览非保密数据，推导所需信息

系统测试——恢复测试（*Recovery Testing*）

- 恢复测试是检验系统从软件或者硬件失败中恢复的能力，即采用各种人工干预方式使软件出错，而不能正常工作，从而检验系统的恢复能力。
- 恢复性测试的例子
  - 当供电出现问题时的恢复
  - 恢复程序的执行
  - 对选择的文件和数据进行恢复
  - 恢复处理日志方面的能力
  - 通过切换到一个并行系统来进行恢复

系统测试——GUI测试（*Graphic User Interface Testing*）

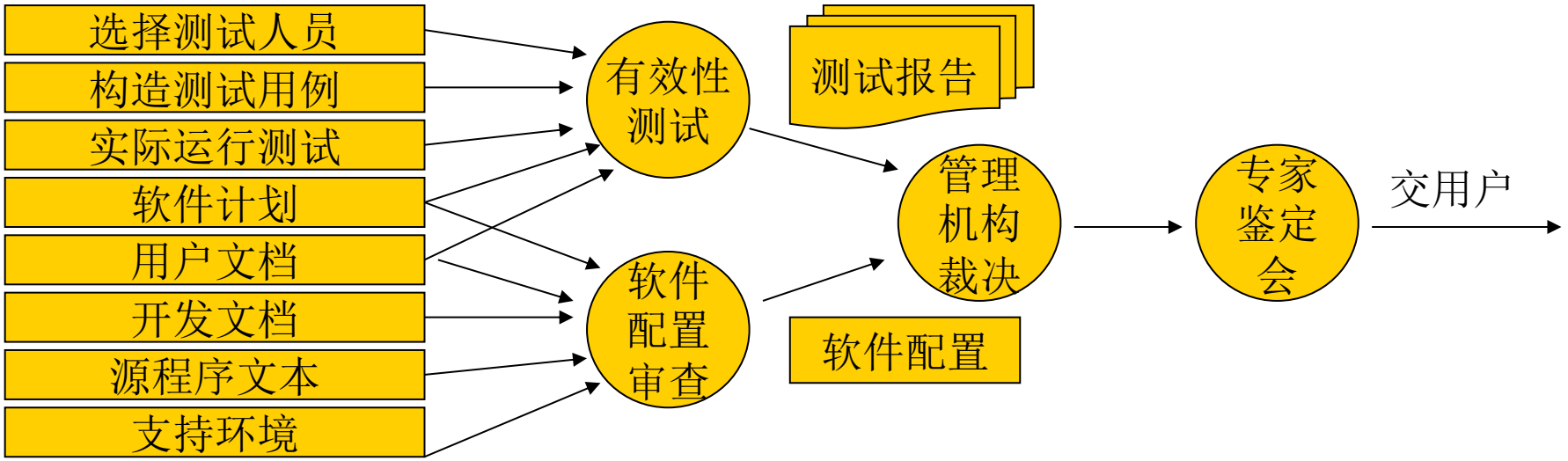
- GUI 测试一是检查用户界面实现与设计的符合情况，二是确认用户界面处理的正确性。
- GUI 测试提倡界面与功能的设计分离，其重点关注在界面层和界面与功能接口层上。
- GUI自动化测试工具
  - WinRunner，QARun，QARobot，Visual Test
- 常用的测试技术
  - 等价类划分、边界值分析、基于状态图方法、错误猜测法

系统测试——安装测试（*Installation Testing*）

- 系统验收之后，需要在目标环境中进行安装，其目的是保证应用程序能够被成功地安装。
- 安装测试应考虑
  - 应用程序是否可以成功地安装在以前从未安装过的环境中?
  - 应用程序是否可以成功地安装在以前已有的环境中?
  - 配置信息定义正确吗?
  - 考虑到以前的配置信息吗?
  - 在线文档安装正确吗?
  - 安装应用程序是否会影响其他的应用程序吗?
  - 安装程序是否可以检测到资源的情况并做出适当的反应?

确认测试

验证软件的功能和性能及其他特征是否与用户的要求一致。



确认测试——验收测试（*Acceptance Testing*）

- 验收测试是以用户为主的测试，一般使用用户环境中的实际数据进行测试。
- 在测试过程中，除了考虑软件的功能和性能外，还应对软件的兼容性、可维护性、错误的恢复功能等进行确认。
- α测试与β测试
  - 产品在正式发布之前，将其提供给一小部分选定的潜在用户进行试用（α测试）和/或提供给更多的代表性用户（β测试）。

冒烟测试

- **冒烟测试**(smoke testing)，通常是指产品到达可提交测度阶段，对主流程或者基本业务的测试。
- 冒烟测试是指测试人员拿到待测软件后，并不立即投入到系统测试用例的执行中，而是先筛选一些基本功能点进行测试，如果筛选出来的基本功能点经过测试没有问题，再进行系统测试。如果有问题，则停止测试，由开发人员将问题解决后再进行系统测试。
- 冒烟测试时间通常尽量在半天内完成，若冒烟测试不通过可以直接打回，拒绝进行第一轮测试。
- 冒烟测试可以由开发工程师完成，也可以由测试团队完成。

灰盒测试

- **灰盒测试**（Grey Box Testing），是介于白盒测试与黑盒测试之间的一种测试，灰盒测试多用于集成测试阶段，不仅需要关注输出、输入的正确性，同时也需要关注程序内部的情况。
- 灰盒测试不会像白盒那样详细和完整，但又比黑盒测试更关注程序的内部逻辑，常常是通过一些表征性的现象、事件、标志来判断内部的运行状态。

mock测试

- **mock测试**就是在测试过程中，对于某些不容易构造或者不容易获取的对象，用一个虚拟的对象（mock对象）来创建以便于测试的测试方法。
- mock注入，就是阻断被测服务与真实服务之间的链路，建立被测服务与mock对象之间的链路过程。mock的本质就是用模拟桩来替换真实的依赖。
- mock对象适用场景：
  - 需要将当前被测试单元独立开来，构造一个独立的环境，不关注被测试单元的依赖对象，只关注被测试单元的功能逻辑。
  - 被测试单元的模块尚未开发完成，而被测单元需要依赖模块的返回值进行后续处理。
  - 被测单元依赖的对象较难模拟或构造比较复杂。

软件故障注入测试

- **软件故障注入测试**（software fault injection testing, SFIT）技术目的是根据一定的故障类型模拟注入相关的故障，以便观察系统的输入与输出并评估系统的健壮性和可靠性，最终避免实际应用中发生软件失效并造成损失。
- 软件故障注入方式按注入时间分为编译故障注入和运行故障注入。
- 通常可注入的故障类型有：

  1）内存错误

  2）处理器错误

  3）通信错误

  4）进程错误，如死锁、活锁、进行循环、进行挂起及使用过多的系统资源等

  5）消息错误，如失去消息、已损坏的消息、无序的消息、信息复制和组件间等待消息时间超时

  6）网络错误

  7）操作系统错误

  8）程序代码错误，如语句、变量等

混沌工程

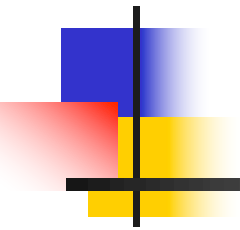
- 混沌工程（Chaos Engineering）定义：
  - 混沌工程是在生产或预生产环境中有意以受控方式造成故障，以了解故障影响并规划更好的防御态势和事件维护策略。[IBM]
  - 混沌工程是一种通过主动注入故障和异常，验证系统在混乱环境下是否具备韧性和容错能力的实践方法。
- 核心思想：
  - 通过主动制造“可控的故障”，提前发现系统的脆弱点，从而提升系统的可靠性和稳定性。
- 主要理念：
  - “通过故障来学习”，即通过故障注入和压力测试，确保系统能够在发生意外事件时依然保持高可用性和稳定性。

（续）

- 与故障注入的区别：
  - 故障注入是“如何制造故障”，关注技术实现；
  - 混沌工程是“为什么制造故障”和“如何通过故障改进系统”，关注系统韧性的系统性验证与改进。
- 与传统测试的区别
  - 传统测试关注“系统是否按预期工作”；
  - 混沌工程关注“系统在异常时如何保持稳定”。
- 混沌工程不是替代传统测试，而是填补了传统测试无法覆盖的“未知风险”领域。两者结合使用，可以构建既正确又高可用的系统。

程序的静态分析方法

- 对程序的静态分析
  - 生成各种引用表
  - 静态错误分析
- 人工测试
  - 桌前检查
  - 代码评审
  - 走查



软件测试

9 面向对象测试

面向对象测试—主要内容

1. OO单元测试
2. OO集成测试
3. 确认测试与系统测试
4. OO软件的测试用例



OO概念对测试的影响

- 封装性和继承性给OO软件开发带来很多好处，同时它又对OO软件测试带来负面影响。
- 类的属性和操作是被封装的，而测试需要了解对象的详细状态。同时，需要考虑当改变数据成员的结构时，是否影响了类的对外接口，是否导致相应外界必须改动。
- 继承不会减少对子类的测试，相反，会使测试过程更加复杂化。当父类与子类的环境不同时，父类的测试用例对子类没有什么使用价值，必须为子类设计新的测试用例。

OO软件测试与传统的软件测试的差异

- 测试对象和基本单位不同
  - 传统软件测试的对象是面向过程的软件，通常使用结构化方法构建，测试的基本单位是模块。
  - OO测试的对象是面向对象软件，采用OO的方法构建，测试的基本单位是类和对象。
- 测试方法和策略不同。
  - 传统软件测试主要采用白盒测试、黑盒测试和路径覆盖等方法。
  - OO测试不仅用传统方法，还采用了各种类测试方法。
  - 集成测试和系统测试的方法和策略也有很大不同，主要采用基于类间协作关系的横向测试和基于类间继承关系的纵向测试。
- 测试的焦点和视角不同
  - 传统单元测试主要关注模块的算法实现和模块接口间数据的传递
  - OO的单元测试主要考察封装在一个类中的方法和类的状态行为。
- 面向对象软件的新特性对测试的影响
  - 封装使得类的属性和状态对外不可见，增加了测试用例生成的难度；继承使得一个函数可能被封装在多个类中，子类可以重新定义父类的行为，需要重新测试；多态性增加了系统运行中的执行路径，增加了测试用例的数量和选取难度。

9.1 OO软件的单元测试

- 封装起来的类和对象是最小的可测试单元。
- 一个类可以包含一组不同的操作，而一个特定的操作也可能定义在一组不同的类中。
- 全面地测试类和对象所封装的属性和操纵这些属性的操作整体。
- 在OO的单元测试中不仅要发现类的所有操作中存在的问题，还要考查一个类与其他的类协同工作时可能出现的错误。

9.2 OO软件的集成测试

- **OOP没有层次的控制结构**，传统的集成测试不能用于OOP中。
- 由于OOP具有动态性，程序的控制流往往难以确定，因此**只能做基于黑盒方法的集成测试**。
- OO集成测试主要关注于系统的结构和内部的相互作用。

OO集成测试的两种方法：

- **基于线程的测试(Thread-based Testing)**
  - 基于线程的测试是指把响应系统一个输入或一个事件所需要的一组类集成起来进行测试。
  - 应当分别集成并测试每个线程，同时为了避免产生副作用再进行回归测试。
- **基于使用的测试(Use-based Testing)**
  - 首先，测试独立类——几乎不使用服务器类的那些类，把独立类都测试完；
  - 接着，测试依赖类——使用独立类的最下层的类。
  - 然后，根据依赖类的使用关系，从下到上一个层次一个层次地持续进行测试，直至把整个软件系统测试完为止。

9.3 OO软件的确认测试与系统测试

- 通过对OO软件单元测试和集成测试，还必须经过规范的确认测试和系统测试。
- OO软件的确认测试或系统测试，与传统的确认测试一样，通过设计测试用例，主要检查用户界面和用户可识别的输出，不再考虑类之间相互连接的细节。

9.4 设计测试用例

1993年，Berard提出了指导OO软件测试用例设计的方法，其要点如下：

- 每一个测试用例都要有一个惟一的标识，并与被测试的一个或几个类相关联起来；
- 每个测试用例都要陈述测试目的；
- 对每个测试用例要有相应的测试步骤，包括被测对象的特定状态，所使用的消息和操作，可能产生的错误及测试需要的外部环境。
- 与传统软件测试（测试用例的设计由软件的输入— 处理— 输出或单个模块的算法细节驱动）不同，面向对象测试关注于设计适当的操作序列以检查类的状态。

设计OO测试用例应注意：

① 继承的成员函数需要测试。一般在下述两种情况下要对成员函数重新进行测试：

- 继承的成员函数在子类中有所改动；
- 成员函数调用了改动过的成员函数。

② 子类的测试用例可以参照父类。

③ 设计测试用例时，不但要设计确认类功能满足的输入，而且还应有意识地设计一些被禁止的例子，确认类是否有不合法的行为产生。

类测试用例设计

一般来说，在设计测试用例时，可参照下列步骤：

① 根据OOD分析结果，选定检测的类，并仔细分出类的状态和相应的行为，以及成员函数间传递的消息和输入输出的界定。

② 确定覆盖标准。

③ 利用结构关系图确定待测试类的所有关联。

④ 构造测试用例，确认使用什么输入来激发类的状态，使用类的什么服务，期望产生什么行为。

基于故障的测试用例设计

- 基于故障的测试（Fault-based Testing）与传统的错误推测法类似，通过对OOA/OOD模型的分析，首先推测软件中可能有的错误，然后设计出最可能发现这些错误的测试用例。
- 基于故障的测试用例有一个很突出的缺点：当功能描述是错误的或子系统间交互存在错误时，基于故障的测试用例就无法发现错误。

基于用例的测试用例设计

- 基于用例（Use-case-based Testing）的测试用例更关心的是用户想做什么而不是软件想做什么，通过用例获取用户要完成的功能，并以此为依据设计所涉及的各个类的测试用例。
- 更具体地说，先搞清楚用户想实现哪些功能？然后去寻找要完成的这些功能，需要哪些类参与，从功能出发，对所确定的这些类及其子类分别设计类测试用例。

类间测试用例设计

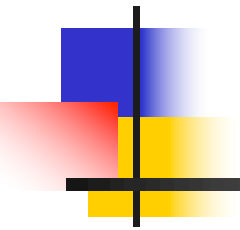
- 在集成面向对象系统阶段，必须对类间协作进行测试，因此测试用例的设计变得更加复杂。
- 通常可以从OOA的对象-关系模型和对象-行为模型导出类间测试用例。
- 测试类协作可以使用随机测试方法和划分测试方法，以及基于情景的测试和行为测试来完成。

多个类测试

- 多个类的划分测试方法有三种：
  - 基于状态的划分：根据类操作改变类状态的能力来划分类操作；
  - 基于属性的划分：根据类操作使用的属性来划分类操作；
  - 基于功能的划分：根据类操作所完成的功能来划分类操作。
- 另外，多类测试还应该包括那些通过发送给协作类的消息而被调用的操作。根据与特定类的接口来划分类操作又是一种划分测试方法。

从行为模型导出测试用例。

- 动态行为模型是由几个状态转换图组成的。
- 根据类的状态图，可以设计出测试该类以及类间的动态行为的测试用例。



软件测试

10 软件测试的挑战

软件测试面临挑战

① **不可能穷尽测试**：路径爆炸问题，这是测试最根本的理论挑战。
- 存在输入组合爆炸、执行路径爆炸、状态空间爆炸，不可能测试所有东西，但必须决定"测哪些就够了"。

② **Test Oracle问题**：不知道什么是对的，这是测试自动化的终极瓶颈。
- 例如：在超大规模分布式系统、物联网 (IoT) 与边缘计算的碎片化等系统中，复杂输出、非确定性系统、没有明确规格

③ **流程与效率的挑战**：速度与质量的平衡
- “左移”与“右移”，以及持续测试压力，测试数据的管理与合成、自动化测试的维护噩梦 (Flaky Tests)

④ **回归测试的规模膨胀**：测试越跑越慢
- 随着系统演进，回归测试用例越来越多，测试时间越来越长。

⑤ **环境的复杂性**：测试环境与生产环境的不一致，是测试失效的常见原因。

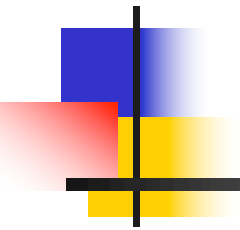
⑥ **非功能性测试的滞后**：性能问题发现太晚
- 功能测试往往先做，性能、安全、可用性等非功能测试往往到后期才做，发现问题时已经很难改。

⑦ **AI系统的测试难题**：不知道错在哪
- 例如：AI与生成式内容存在无预期输出、不可解释性、数据漂移、对抗攻击、偏见问题

测试挑战的层次

- 理论层
  - 不可能穷尽测试、Oracle问题
- 技术层
  - 回归测试膨胀、环境复杂、AI系统
- 过程层
  - 非功能测试滞后、缺陷分布不均
- 组织层
  - 测试与开发的张力

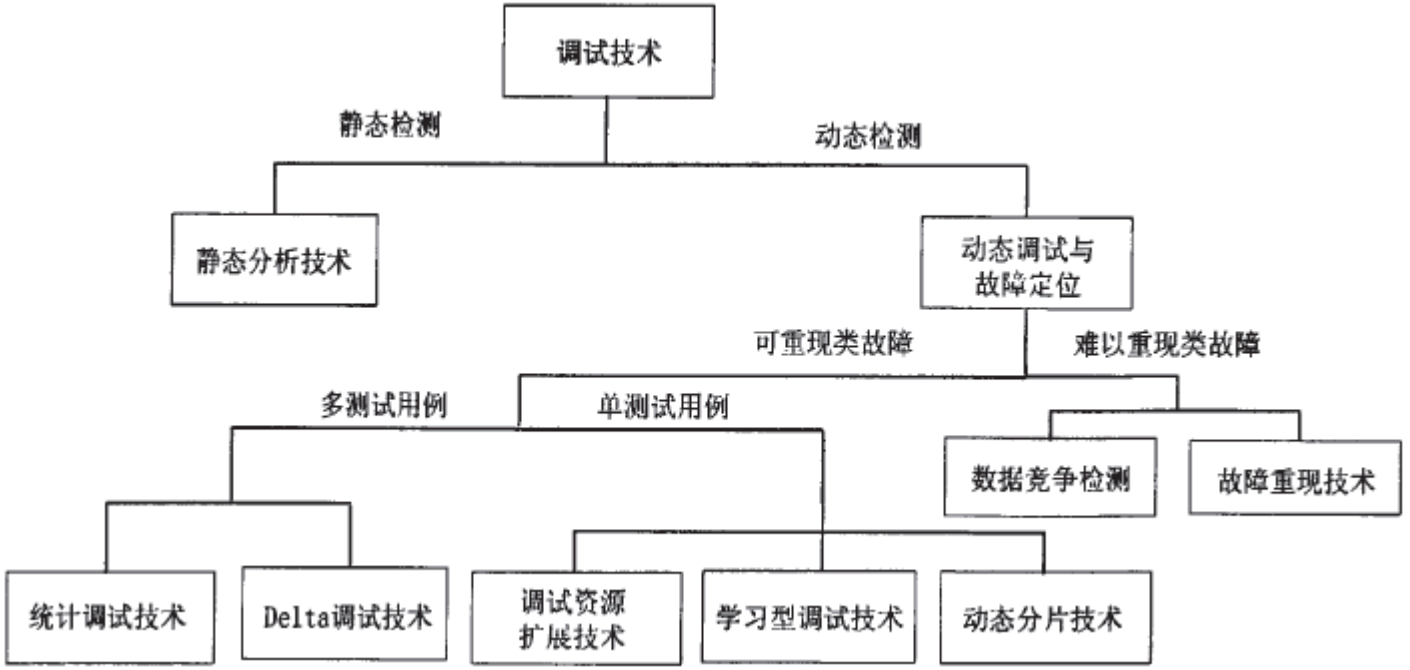
- 当前软件测试的核心矛盾是：软件系统的**复杂度和迭代速度呈指数级增长**，而人类测试资源的线性增长无法匹配。
- 软件测试的本质是**"在有限的时间内，用有限的资源，尽可能多地发现缺陷，并提供质量信心"**。



软件测试

11 软件测试工具

软件调试技术分类


调试技术
静态检测
动态检测
静态分析技术
动态调试与
故障定位
可重现类故障
难以重现类故障
多测试用例
单测试用例
数据竞争检测
故障重现技术
统计调试技术
Delta调试技术
调试资源
扩展技术
学习型调试技术
动态分片技术

软件测试工具主要应用在下列方面：

- 静态分析（Static Analyzer）
- 代码审计（Code Auditor）
- 断言处理（Assertion Processor）
- 测试文件生成（Test File Generator）
- 测试数据生成（Test Data Generator）
- 测试验证（Test Verifier）
- 输出比较（Output Comparator）

思考题

① 保证绝对正确的程序可行吗？说明理由。

② 如何计算复杂度

③ 测试与软件开发周期的关系

④ 四种测试策略的内容

习题

习题二：A:(2) B:(5) C:(1) D:(1) E:(4)

在设计测试用例时，__A__ 使用的最多的一种黑盒方法。黑盒测试中，等价类划分方法设计测试用例步骤是：

1.根据输入条件把输入数据分成若干有效等价类和无效等价类。

2.设计一个用例，使其覆盖__B__ 尚未被覆盖的有效等价类，重复这一步，直到所有的有效等价类均被覆盖。

3.设计一个用例，使其覆盖__C__ 尚未被覆盖的无效等价类，重复这一步，直到所有的无效等价类均被覆盖。

因果图方法是根据__D__ 之间的因果关系来设计用例的。

在实际中，纠正了程序中的错误后，还应测试原先测试过的测试用例，对修改过的程序重新测试，这种测试称为__E__ 。

A：(1)等价类划分　(2)边值分析　(3)因果图　(4)判定表

B、C：(1)1个　(2)7个左右　(3)一半　(4)尽可能少　(5)尽可能多　(6)全部

D：(1)输入与输出　(2)设计和实现　(3)条件与结果　(4)主程序与子程序

E：(1)验收测试　(2)强度测试　(3)系统测试　(4)回归测试

习题

习题三： A:(4) B:(3) C:(1) D:(2) E:(3)

__A__ 在实现阶段进行，它所依据的模块功能描述和内部细节以及测试方案应在__B__ 阶段完成，目的是发现编程错误。

__C__ 所依据的模块说明书和测试方案应在__D__ 阶段完成，它能发现设计错误。

__E__ 应在模拟的环境中进行强度测试的基础上进行，测试计划应在软件需求分析阶段完成。

A: (1)用户界面测试 (2)输入输出测试 (3)集成测试 (4)单元测试

B: (1)需求分析 (2)概要设计 (3)详细设计 (4)结构设计

C: (1)集成测试 (2)可靠性测试 (3)系统性能测试 (4)强度测试

D: (1)编程 (2)概要设计 (3)维护 (4)详细说明

E: (1)过程测试 (2)函数测试 (3)确认测试 (4)逻辑路径测试

课堂练习——白盒测试用例设计

问题描述：

- 程序要求：10个铅球中有一个假球（比其他铅球的重量要轻），用天平三次称出假球。
- 程序设计思路：第一次用天平分别称5个球，判断轻的一边有假球。拿出轻的5个球，称其中4个球，两边分别放2个球；如果两边同重，则剩下的球为假球；若两边不同重，拿出轻的两个球称第三次，轻的为假球。

要求：

① 算法设计，可以用C/C++、伪代码和流程图表示。

② 用白盒测试技术的条件组合覆盖，为程序设计足够多的测试用例。